

UAB

Universitat Autònoma de Barcelona

Faculty of Science

Design, Integration, and Validation of a Modular
Language-Model-Enabled Interaction and Execution
Stack for the NAO Robot

Juan David Bendek Williamson

Supervised by Raquel Ros

Institution / Lab: IIIA-CSIC / Universitat Autònoma de Barcelona

*Thesis submitted as a requirement for the degree of
MSc in Modelling for Science & Engineering*

Academic year: 2025–2026

Date: July 14, 2026

Abstract

Large language models have shown strong potential for instruction understanding and high-level reasoning in human-robot interaction. Their integration into a robot nevertheless requires more than a planner call: dialogue state, symbolic grounding, capability descriptions, execution admission, robot-facing skills, feedback, and user-facing reporting must remain coherent across a distributed runtime. This thesis presents the design, integration, and validation of a modular language-model-enabled interaction and execution stack for the NAO robot.

The system separates dialogue management, semantic routing, symbolic grounding, task planning, deterministic orchestration, and skill execution. A chatbot interprets each turn and selects a conversational, knowledge, or execution route. A planner generates structured plans over a reviewed skill registry, while an orchestrator admits requests, validates plans, dispatches real or simulated skills, and returns typed execution feedback. A knowledge and perception path provides compact grounded context without allowing planning-time state to stand in for execution evidence.

The contribution is an inspectable NAO interaction stack and a contract-based validation method for its principal boundaries. The validation follows complete user turns as well as isolated planner and deterministic fake-skill profiles. It examines route correctness, plan validity, entity and role preservation, execution lineage, symbolic postconditions, failure handling, and exactly-once user-facing closure. Direct atomic execution is retained as an orchestrator control rather than treated as a causal baseline for multi-step planning.

Keywords: Human-Robot Interaction, ROS 2, Large Language Models, Task Planning, NAO Robot, Symbolic Grounding, Deterministic Execution, Runtime Validation.

Acknowledgements

This page is reserved for the final acknowledgements.

Contents

Abstract	i
Acknowledgements	ii
List of Figures	vii
List of Tables	viii
List of Abbreviations	x
1 Introduction	1
1.1 Language-Guided Robot Interaction and Execution	1
1.2 Problem Statement	2
1.3 Research Questions	2
1.4 Objectives	2
1.5 Document Structure	3
2 Background and Literature Review	4
2.1 Human-Robot Interaction	4
2.2 ROS 2 for Distributed Robot Software	4
2.3 Reusable HRI Representations and Interfaces	5
2.4 The NAO Robot Platform	5
2.5 LLMs as High-Level Planners	5
2.6 Embodied Multimodal Models, VLMs, and VLAs	6
2.7 Symbolic State, Knowledge Bases, and Grounded Context	7
2.8 Execution Monitoring, Replanning, and Clarification	7
2.9 Literature Gap	7

3	System Requirements and Architecture	9
3.1	Functional Requirements	9
3.2	Design Principles	9
3.3	High-Level ROS 2 Architecture	10
3.3.1	Runtime Flow	11
4	Runtime Contracts	13
4.1	Runtime Ownership Rules	13
4.2	Runtime Contract Overview	13
4.3	Contract 1: Chatbot Turn Output	14
4.4	Contract 2: Planner Request	16
4.5	Contract 3: Grounded Context	18
4.6	Contract 4: Shared Skill Entry	19
4.7	Grounded Context Pipeline	21
4.8	Planner LLM Internals	21
4.9	Contract 5: Planner Output	22
4.10	Contract 6: Skill Execution Result	24
4.11	Contract 7: Execution Feedback	24
4.12	Contract 8: Planner Dialogue Act	25
4.13	Contract 9: RDF Knowledge State and Raw Evidence	26
5	Implementation: ROS 2 Nodes and APIs	28
5.1	Runtime Flow	28
5.2	ROS Nodes	31
5.2.1	<code>dialogue_manager</code>	31
5.2.2	<code>chatbot_llm</code>	31
5.2.3	<code>planner_llm</code>	33
5.2.4	Goal-Keyed Planning-State Helper	34
5.2.5	Skill Registry (<code>skill_common</code>)	34
5.2.6	<code>nao_orchestrator</code> and the Execution Boundary	35
5.3	Deterministic Guards and Bounded Fallbacks	37
5.4	World State and Scene Grounding	38
5.5	Fake Skills and Simulation	39

5.6	Models and Runtime Configuration	40
6	Validation Methodology	42
6.1	Validation Scope and Unit of Analysis	42
6.2	Validation Levels and Supported Claims	42
6.3	Execution Profiles	43
6.4	Validation Goals and Acceptance Rules	44
6.5	Full Runtime Questionnaire	46
6.6	Multi-Step Deterministic Fake-Skill Matrix	47
6.7	Robustness Set and Repetition	48
6.8	Interface and Architecture Checks	49
6.9	Reset, Repetition, and Isolation	49
6.10	Metrics and Denominators	50
6.11	Runtime Review Score	51
6.12	Research-Question Evidence Map	52
6.13	Evidence Bundle	53
6.14	Threats to Validity	54
7	Results	55
7.1	Selected Evidence Sets	55
7.2	Trajectory and Semantic Contract Results	57
7.3	Focused Post-Hardening Results	57
7.4	Full Integrated Runtime Results	58
7.5	Deterministic Fake-Skill and Replan Results	59
7.6	Strict KB-Effect and Reporting Audit	59
7.7	Alternate-Model Ablation	60
7.8	Interaction and Runtime-Contract Preservation	60
7.9	Research-Question Synthesis	61
8	Discussion	63
8.1	Architectural Findings	63
8.2	Harness Engineering and Failure Attribution	63
8.3	Prompt Packs and Skill Contracts as Runtime Policy	65

8.4	Planner Behaviour and Feedback	65
8.5	Grounding, Identity, and World State	66
8.6	Execution Evidence and Truthful Closure	67
8.7	ROS 2 and HRI Modularity	67
8.8	Validation Implications	68
9	Limitations and Future Work	69
9.1	Current Limitations	69
9.2	Future Work	69
10	Conclusion	70
A	Full Runtime Contracts	72
A.1	Runtime Skill Inventory	72
B	Launch Commands	75
C	Validation Log Template	76
	Bibliography	77

List of Figures

3.1	High-level layered architecture. Dialogue interpretation and symbolic grounding provide parallel inputs to the planning layer. The chatbot layer also consumes symbolic scene context. The planner proposes structured skill plans, while the execution layer validates and dispatches skills before returning feedback for replanning or dialogue.	11
3.2	Runtime message and responsibility sequence from user input to spoken response. The colour coding follows Figure 3.1: dialogue components are blue, grounding and evidence are orange, planning is green, and execution is purple.	12
4.1	Grounded-context pipeline. Scene and KB evidence are projected by a grounded perception helper into a compact <code>grounded_context</code> for <code>chatbot_llm</code> and <code>planner_llm</code> . Skills receive planner-approved arguments through execution, while also retaining access to raw scene and KB evidence for live requery and validation before reporting outcomes.	21
5.1	Full-stack implementation of the modular NAO interaction and execution system. Dialogue components are blue, grounding and evidence components orange, planning components green, execution components purple, and skills or adapters neutral grey. The <code>interaction_sim</code> and KB-fixture paths provide controlled validation inputs. The dashed red World Model Enricher box denotes future work rather than an implemented runtime component.	30
5.2	Internal <code>chatbot_llm</code> path. The node owns semantic interpretation and candidate planner metadata, but not plan construction, dispatch, or speech execution.	33
5.3	Internal <code>planner_llm</code> path, from admitted request through validated plan envelope and feedback-driven goal-state handling.	34
5.4	The <code>nao_orchestrator</code> admission and execution boundaries. All effects cross deterministic validation and retain goal and plan lineage.	36
5.5	Combined perception and symbolic-grounding pipeline. Compact projections serve dialogue and planning, while raw and fresh evidence remains at its source interface.	39

List of Tables

3.1	Functional requirements for the proposed NAO interaction and execution stack.	9
4.1	Runtime contract map and ownership in the active dialogue, planning, and execution stack.	14
4.2	Fields in Contract 1: chatbot turn output.	15
4.3	Fields in Contract 2: minimal planner-facing request.	16
4.4	Fields in Contract 3: grounded context.	19
4.5	Shared skill-entry fields used by planning and execution.	20
4.6	Fields in Contract 5: planner output.	23
4.7	Principal RDF-style predicate families used by the runtime KB.	26
5.1	Planner-visible runtime skills and their current execution profiles.	35
5.2	Fake-skill outcome-policy resolution order.	39
5.3	Principal integrated launch profiles used by the implementation.	40
6.1	Validation levels and supported claims.	43
6.2	Evaluation profiles and their validation scope.	44
6.3	Validation goals, acceptance rules, and evidence sources.	45
6.4	Frozen twenty-one-case full-runtime questionnaire.	46
6.5	Canonical multi-step deterministic fake-skill manifest.	48
6.6	Frozen five-case robustness set.	49
6.7	Components of the run-level reviewer score.	52
6.8	Research questions, primary profiles, and acceptance evidence.	53
7.1	Run-level and diagnostic evidence sets.	56
7.2	Trajectory and semantic findings from evidence set F13.	57
7.3	Full-runtime results for evidence set F28.	58

7.4	Deterministic fake-skill policy results for evidence set F08.	59
7.5	Alternate-model ablation results for evidence set F11.	60
7.6	Interaction-interface and runtime-contract results.	61
7.7	Qualified answers to the research questions from the recovered evidence.	62
8.1	Representative failure shapes and the architectural boundaries needed to diagnose them.	64
A.1	Compressed runtime skill inventory and registry values.	73
C.1	Validation log template.	76

List of Abbreviations

Abbreviation	Meaning
API	Application Programming Interface
ASR	Automatic Speech Recognition
HRI	Human-Robot Interaction
HTN	Hierarchical Task Network
JSON	JavaScript Object Notation
KB	Knowledge Base
LLM	Large Language Model
NAO	Humanoid robot used as the main experimental platform
PDDL	Planning Domain Definition Language
ROS	Robot Operating System
ROS4HRI	ROS conventions and interfaces for Human-Robot Interaction
TFM	Trabajo de Fin de Máster
TTS	Text-to-Speech
VLA	Vision-Language-Action model
VLM	Vision-Language Model
WME	World Model Enricher

Chapter 1

Introduction

1.1 Language-Guided Robot Interaction and Execution

Human-robot interaction (HRI) requires a robot to connect communication with perception, task reasoning, and embodied action. Automated planning provides one part of this problem. Classical languages such as the Planning Domain Definition Language (PDDL) make actions, preconditions, effects, initial states, and goals explicit (McDermott et al. 1998). Hierarchical task network methods further describe how high-level tasks can be decomposed into executable subtasks (Erol et al. 1994; Nau et al. 2003; Georgievski et al. 2014). Language-model-assisted planning revisits the connection between flexible instructions and grounded action structures (Liu et al. 2023a; Valmeekam et al. 2023; Ahn et al. 2022).

Large language models (LLMs) can transform user utterances into structured intent payloads, infer task goals, and propose multi-step action decompositions. This flexibility is useful because users rarely speak in planner-ready commands. They ask questions, refer to visible people or objects, change goals, and expect responses that remain consistent with the interaction history. LLMs are not inherently grounded in the robot’s live state, available skills, or execution constraints. A linguistically plausible response may select the wrong route, refer to an unavailable entity, propose an unsupported action, or report success without corresponding execution evidence.

This thesis addresses that problem across the complete NAO stack. The implemented system combines dialogue lifecycle management, LLM-based semantic routing, compact symbolic grounding, structured task planning, deterministic orchestration, robot-facing skills, and feedback-driven user communication. The language model participates in dialogue and planning, while ROS 2 interfaces, reviewed skill contracts, and an orchestrator retain control over admissible execution. The focus is therefore the integrated interaction and execution stack rather than the planner in isolation.

1.2 Problem Statement

Five recurring problems define the scope of this thesis:

- integrations may be tightly coupled to one robot or task domain;
- prompt-only action generation may lack deterministic execution validation;
- dialogue, planning, perception, knowledge, and execution ownership may be ambiguous;
- composite instructions may lose entity roles or ordering as they cross component boundaries;
- clarification, replanning, failure recovery, and evidence-based reporting may be incomplete.

The central problem addressed in this thesis is therefore:

How can a modular NAO robot stack integrate language-based dialogue, symbolic grounding, task planning, deterministic execution, and feedback while preserving inspectable ownership boundaries and evidence-based user communication?

1.3 Research Questions

The work is guided by five research questions. Each question is linked to evidence defined in Chapter 6.

1. **RQ1:** Can the integrated language-guided stack translate natural-language requests into coherent multi-step plans and ordered skill execution for the supported task families?
2. **RQ2:** To what extent does compact symbolic grounding preserve the objects, people, locations, and interaction roles required from dialogue interpretation through execution and reporting?
3. **RQ3:** Does a reviewed skill registry, combined with deterministic admission and validation, reject unsupported skills, malformed arguments, invalid lineage, and robot-specific calls before dispatch?
4. **RQ4:** How does typed execution feedback affect the stack’s ability to recover from failed actions, request clarification, revise a plan, or stop safely when the current plan cannot continue?
5. **RQ5:** Which ownership and interface boundaries are required to keep language-guided interaction, planning, execution, and user-facing reporting inspectable within the evaluated NAO stack?

1.4 Objectives

The main objective is to design, integrate, and validate a modular language-model-enabled interaction and execution stack for NAO. This objective is divided into the following imple-

mentation and validation goals:

1. integrate dialogue management, semantic routing, symbolic grounding, task planning, deterministic orchestration, and skill execution through explicit ROS 2 interfaces;
2. define runtime contracts for chatbot output, planner admission, grounded context, skill descriptions, plans, results, execution feedback, dialogue acts, and raw evidence;
3. expose robot capabilities through a reviewed skill registry rather than direct model-generated robot APIs;
4. support conversational, knowledge-query, atomic execution, and composite execution turns without transferring speech or execution ownership to an LLM;
5. constrain candidate plans through deterministic validation before robot actions are dispatched;
6. return typed execution evidence to the planning and dialogue paths so that failure, ambiguity, replanning, and truthful reporting remain observable;
7. validate the integrated stack through full user turns, planner-isolated probes, controlled fake-skill policies, symbolic postcondition checks, and trace-based acceptance rules.

1.5 Document Structure

The thesis is organised as follows:

1. Chapter 2 reviews HRI, ROS 2, language-based planning, multimodal models, symbolic grounding, and feedback-driven execution.
2. Chapter 3 defines the system requirements, responsibility boundaries, and high-level runtime architecture.
3. Chapter 4 specifies the payloads and evidence ownership shared between the stack components.
4. Chapter 5 maps those abstract responsibilities and contracts to the implemented ROS 2 nodes, skills, launch profiles, and deterministic guards.
5. Chapter 6 defines the validation scope, case manifests, profiles, metrics, acceptance rules, and evidence bundle.
6. Chapter 7 reports the observed trajectory, semantic, failure-injection, KB-effect, model-ablation, and ownership results.
7. Chapter 8 interprets the findings with respect to the research questions and related work.
8. Chapters 9 and 10 present limitations, future work, contributions, and conclusions.

Chapter 2

Background and Literature Review

2.1 Human-Robot Interaction

Human-robot interaction (HRI) studies how people and robots communicate, coordinate tasks, and share physical or social space. In service and social robotics, an interaction cannot be reduced to command recognition. The robot must associate speech and other social signals with people, objects, prior turns, and the state of an embodied task. It must also communicate uncertainty and failure in a manner that is understandable to the user. Surveys of HRI and socially interactive robots therefore describe a multidisciplinary problem spanning robotics, artificial intelligence, perception, dialogue, interaction design, and safety (Fong et al. [2003](#); Goodrich et al. [2007](#)).

These concerns make responsibility boundaries important. Dialogue management must preserve turn and participant context. Perception must retain the provenance and freshness of observations. Planning must operate over capabilities the robot can actually invoke, while execution must reject unsupported or unsafe work. A user-facing response must remain connected to the evidence produced by these layers. The architecture studied in this thesis treats those responsibilities as collaborating components rather than one end-to-end language policy.

2.2 ROS 2 for Distributed Robot Software

The Robot Operating System (ROS) provides the communication infrastructure used to distribute these responsibilities across robot software. Nodes exchange data through typed topics, services, and actions. Topics carry streamed observations and state, services handle bounded request-response operations, and actions support long-running or cancellable work with feedback. ROS 2 extends this model with middleware and deployment mechanisms for distributed robotic systems (Quigley et al. [2009](#); Macenski et al. [2022](#)).

This communication model does not itself define HRI semantics or guarantee correct execution. Its value for this thesis lies in the ability to assign ownership to observable interfaces. Language

models can remain at dialogue and task-planning boundaries, while robot actions, knowledge queries, lifecycle state, and feedback retain typed ROS 2 owners. The same separation supports controlled replacement of real robot skills with deterministic simulated outcomes during validation.

2.3 Reusable HRI Representations and Interfaces

Reusable HRI systems require consistent representations of people and their observable signals. Mohamed et al. (2020) identify the absence of a standard human representation as a barrier to composing perception and interaction pipelines. Their ROS4HRI proposal defines conventions for bodies, faces, voices, groups, and related social signals. This thesis uses those conventions and compatible packages at the public person and speech boundaries; it does not treat ROS4HRI as the planning or execution framework of the system.

In a modular HRI system, information passes through several levels of abstraction. Sensor and speech streams first become tracked observations. Tracking and semantic sources associate those observations with people, objects, and interaction groups. A knowledge representation then exposes bounded entities and relations to dialogue and planning. User intent and grounded roles become candidate skill plans, and admitted plans become typed robot actions whose results feed the knowledge and dialogue paths. Each transition removes detail, so it must preserve the identifiers, roles, freshness, and provenance required by the next layer. An error at one transition can otherwise appear as a prompt, planning, or execution failure downstream.

2.4 The NAO Robot Platform

The physical platform used in this thesis is NAO, a programmable humanoid robot originally developed by Aldebaran Robotics. Gouaillier et al. (2008) describe NAO as an autonomous humanoid platform for research, education, and RoboCup experimentation. Its speech capabilities, articulated body, camera, and established academic use make it suitable for studying spoken interaction, person and object grounding, gaze-oriented actions, motion, and feedback-driven task execution. NAO is the embodied platform in the evaluated system, while higher-level components depend on reviewed skills and ROS 2 interfaces rather than direct model-generated NAOqi calls.

2.5 LLMs as High-Level Planners

Large language models have been studied as high-level planners because they encode procedural and semantic knowledge that can be elicited from natural-language instructions. Huang et al. (2022b) show that language models can decompose embodied tasks into action-like steps, while also exposing the difficulty of mapping unconstrained language to admissible actions. A

linguistically reasonable plan can remain impossible, unsafe, or unsupported by the available robot.

Several approaches ground model proposals in a restricted action set. SayCan combines language-model task knowledge with affordance scores from pretrained skills (Ahn et al. 2022). Code as Policies generates policy code over a supplied API, allowing feedback loops and spatial computations while remaining dependent on reviewed callable primitives (Liang et al. 2022). These approaches motivate the use of an explicit skill registry in this thesis. The model plans over declared capabilities rather than arbitrary natural-language actions.

Reasoning-and-acting methods also show the value of external evidence. ReAct interleaves language reasoning with actions that query tools or environments (Yao et al. 2022). Inner Monologue incorporates perception, success detectors, and human feedback into embodied planning (Huang et al. 2022c). Reflexion stores verbalised feedback as experience across trials without modifying model weights (Shinn et al. 2023). The system in this thesis adopts a narrower runtime loop: a planner emits a structured skill plan, an orchestrator validates and dispatches it, skills return typed results, and subsequent planning or dialogue acts are conditioned on that feedback.

Prompt instructions alone cannot guarantee valid skills, arguments, grounding, or execution order. The implemented stack therefore combines model output with schema validation, registry lookup, argument checks, lineage checks, and deterministic dispatch. The distinction is between using an LLM as one task-reasoning component and allowing it to become the controller of the robot.

2.6 Embodied Multimodal Models, VLMs, and VLAs

Vision-language models (VLMs) connect visual observations with language, while vision-language-action models (VLAs) also map visual and language inputs to robot actions. BLIP-2 connects frozen vision encoders and language models through a lightweight transformer (Li et al. 2023). LLaVA uses visual instruction tuning to support image-conditioned interaction (Liu et al. 2023b). These models show how visual evidence can be transformed into language-compatible representations without themselves defining a complete robot runtime.

Embodied multimodal models extend this direction. PaLM-E incorporates visual and continuous sensor inputs and evaluates them on embodied tasks (Driess et al. 2023). RT-2 represents robot actions as tokens and co-trains on web and robot data (Brohan et al. 2023). OpenVLA provides an open-source VLA trained on a broad collection of robot demonstrations (Kim et al. 2024). This thesis does not train an end-to-end VLA policy. Visual and knowledge evidence instead become compact symbolic context before reaching the dialogue and planning components. Richer spatial or multimodal grounding remains a future extension.

2.7 Symbolic State, Knowledge Bases, and Grounded Context

High-level robot reasoning requires a representation of the world. Raw perception streams are detailed and transient, while unstructured language summaries may omit identity, freshness, or relation information. A symbolic knowledge base (KB) provides one bridge between these representations. KnowRob represents objects, actions, environments, and task knowledge for robot reasoning (Tenorth et al. 2013; Tenorth et al. 2017). RoboBrain similarly combines multiple knowledge sources to support grounding, perception, and planning (Saxena et al. 2014). Other work studies how interaction can help a robot acquire environmental knowledge and resolve referring expressions (Bastianelli et al. 2013; Grassi et al. 2021).

The implementation follows this principle at bounded scale. Detector-derived observations, person state, and selected KB statements are projected into compact grounded context. The projection retains stable handles, semantic type, interaction role, visibility, and a bounded relation set. Dialogue and planning can use those entities without ingesting unrestricted detector payloads or RDF rows. Skills may still query fresh source evidence at execution time. This separation limits prompt growth and prevents planning-time state from being presented as proof of a later physical effect.

2.8 Execution Monitoring, Replanning, and Clarification

Robot planning cannot stop at plan generation. Actions fail, targets disappear, a user may change the goal, and safe continuation may require clarification. Feedback-oriented language-agent and embodied-planning work supports this closed-loop view (Huang et al. 2022c; Yao et al. 2022; Shinn et al. 2023).

The evaluated stack treats execution feedback as a typed runtime contract. A failed or ambiguous result can stop dispatch, request user input, or return control to the planning component. The orchestrator does not create replacement plans, and the planner does not execute skills. Clarification follows the same ownership path: the planning component emits a structured communication request and the dialogue component realises the user-facing utterance. This keeps recovery observable and avoids inventing missing world facts.

2.9 Literature Gap

Prior work establishes the main ingredients separately. Language models can decompose tasks into action-like sequences (Huang et al. 2022b); affordance and API grounding can restrict proposals to available capabilities (Ahn et al. 2022; Liang et al. 2022); multimodal systems can connect vision, language, and action (Driess et al. 2023; Brohan et al. 2023; Kim et al. 2024); symbolic systems can support robot knowledge and grounding (Tenorth et al. 2013; Saxena et al. 2014); and feedback can condition later reasoning (Huang et al. 2022c; Yao et al.

[2022](#)).

The gap addressed here is the integration and validation of these responsibilities within one modular NAO stack. The contribution combines dialogue lifecycle, semantic route selection, compact symbolic grounding, skill-based planning, deterministic admission and dispatch, typed execution feedback, and evidence-based reporting. It occupies a middle position between fixed intent-to-skill mappings and end-to-end language or VLA policies. The stack can express composite tasks and react to feedback, while exposing contracts that permit each transition to be inspected independently.

Chapter 3

System Requirements and Architecture

3.1 Functional Requirements

Table 3.1 lists the functional requirements for the complete interaction and execution stack. They cover natural-language ingress, dialogue routing, symbolic grounding, planning, deterministic execution, feedback, and observability. Their implementation owners are introduced in Section 3.3 and detailed in Chapter 5.

ID	Requirement
FR1	The system shall accept natural-language user input through the dialogue stack.
FR2	The system shall distinguish dialogue-only turns from execution-eligible turns.
FR3	The system shall convert execution-eligible user turns into admitted planner requests before invoking the planner.
FR4	The planner shall generate structured skill plans over an explicit skill registry.
FR5	The orchestrator shall validate planner outputs before any robot skill is dispatched.
FR6	The system shall publish execution feedback after relevant plan and step/skill events.
FR7	The planner shall support asking for clarification, replanning and plan failure when execution feedback indicates that the current plan cannot continue safely.
FR8	The system shall expose traces, logs, or runtime messages suitable for validation of the stack.

Table 3.1: Functional requirements for the proposed NAO interaction and execution stack.

3.2 Design Principles

Four responsibility principles guide the implementation of Table 3.1. They define the component roles before the corresponding ROS 2 package names are introduced.

1. **Dialogue remains distinct from execution.** Greetings, conversation, and questions about the robot’s known state remain non-executable unless the user explicitly requests an action.
2. **The planner proposes structured skill plans.** It selects from the reviewed registry and uses grounded scene context, but does not execute skills, speak directly, or call robot-specific APIs.
3. **The orchestrator remains the execution gate.** It admits execution requests, validates plans, dispatches skills, and publishes typed execution feedback.
4. **Skills own fresh execution evidence.** Perception and execution skills query the knowledge base (KB) or robot state when required. Planning-time context cannot establish that a later action succeeded.

3.3 High-Level ROS 2 Architecture

The responsibilities above are implemented by concrete ROS 2 packages. This section introduces the node names used in later chapters for contracts, source ownership, and runtime evidence. The architecture keeps dialogue, semantic routing, grounding, planning, deterministic execution, and feedback in separate components.

The main nodes and shared representations are:

- **dialogue_manager** acts as the central dialogue ownership component, maintaining conversational state, coordinating turn-taking, managing interaction flow, and serving as the integration point between user-facing dialogue and downstream planning services.
- **chatbot_llm** is responsible for robot utterance generation, intent declaration and goal extraction during planner-relevant turns. It transforms natural language interactions into structured representations that depict the nature of the interaction. **Dialogue** and **knowledge query** turns remain conversational, **execution** turns trigger a planner request.
- **planner_llm** performs proactive task planning over the available skill registry, normalized user intents, dialogue context, and current robot world state. The planner reasons over high-level objectives, decomposes user requests into executable actions, and generates candidate plans that align with both robot capabilities and current world state constraints.
- **nao_orchestrator** provides deterministic plan validation and skill execution management. It verifies planner outputs against available skills and execution constraints, coordinates skill invocation, monitors execution progress, and emits structured execution feedback that can be consumed by both the dialogue and planning layers.
- **grounded_context** is the compact KB-to-language JSON projection defined by Contract 3. It combines selected knowledge statements, the current scene summary, and tracked-person state into role-separated entities and bounded relations for the chatbot and planner.

- Skill servers expose typed execution interfaces over lower-level ROS components and robot capabilities. They encapsulate perception, navigation, manipulation, motion, and communication actions. The orchestrator invokes these interfaces after plan validation, and their results provide fresh evidence for feedback, reporting, and verified KB effects.

Figure 3.1 shows how the packages are grouped into dialogue, grounding, planning, and execution layers. It also shows the intent, context, plan, and feedback paths that connect them.

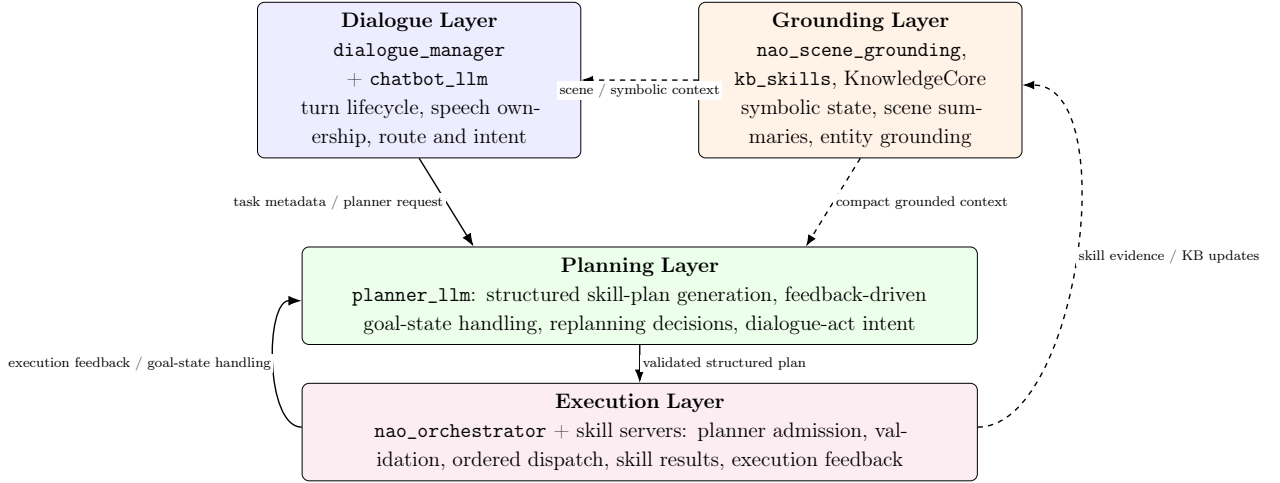


Figure 3.1: High-level layered architecture. Dialogue interpretation and symbolic grounding provide parallel inputs to the planning layer. The chatbot layer also consumes symbolic scene context. The planner proposes structured skill plans, while the execution layer validates and dispatches skills before returning feedback for replanning or dialogue.

3.3.1 Runtime Flow

Figure 3.2 expands the layered view into the normal execution sequence. The numbered stages distinguish user-turn ownership, planner admission, candidate plan generation, deterministic dispatch, fresh skill evidence, and the return path for replanning or user communication.

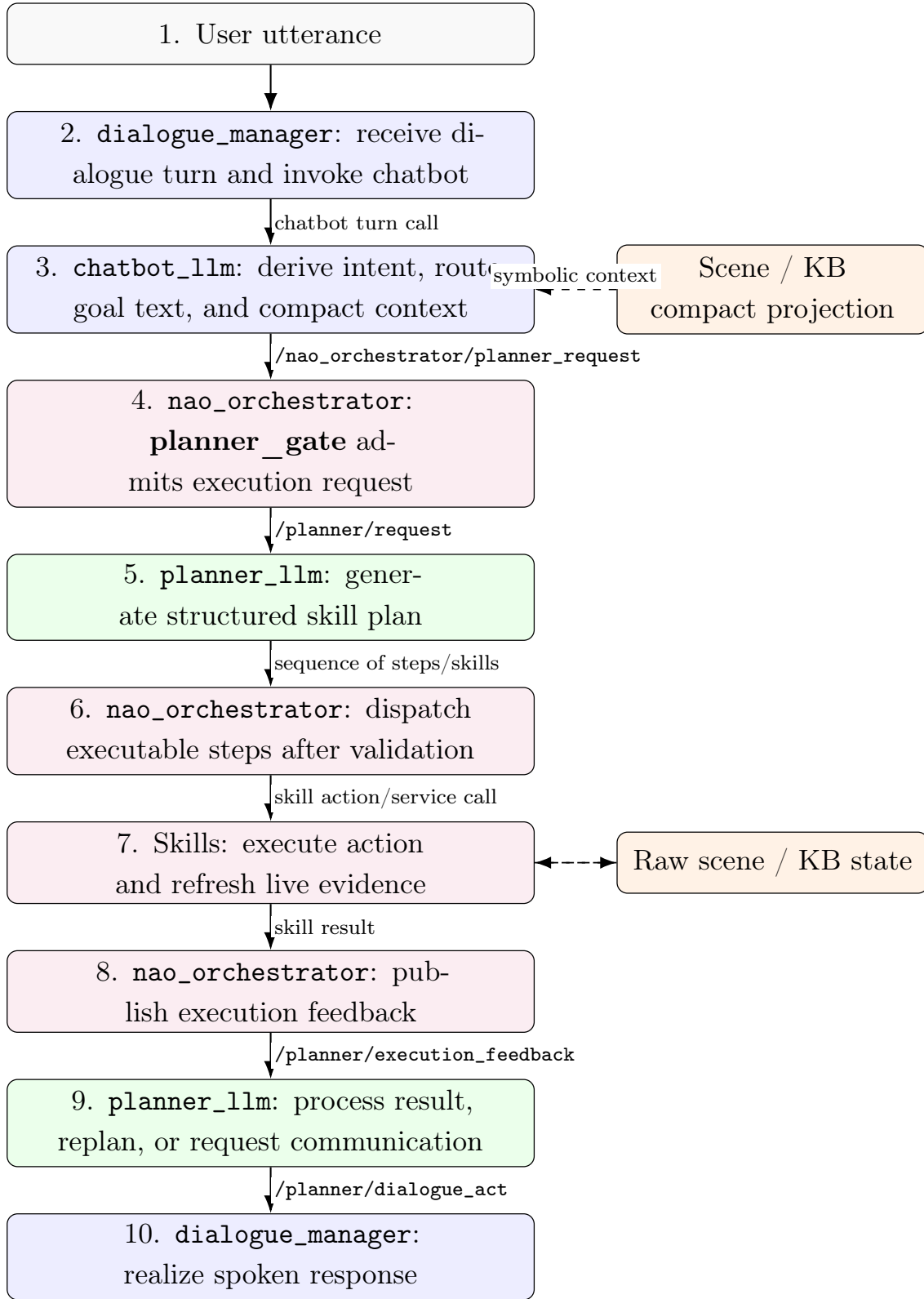


Figure 3.2: Runtime message and responsibility sequence from user input to spoken response. The colour coding follows Figure 3.1: dialogue components are blue, grounding and evidence are orange, planning is green, and execution is purple.

Chapter 4

Runtime Contracts

Runtime contracts define the data exchanged between dialogue interpretation, planner admission, grounding, plan generation, execution, feedback, and speech realization. Each contract names the payload, its producer and consumer, and the component responsible for validation. This keeps the LLM-facing paths inspectable while robot execution remains under deterministic control. The chapter first states the ownership rules, then describes the interfaces and principal payload fields.

4.1 Runtime Ownership Rules

Five ownership rules determine how the contracts are used. Carrying a structured payload does not transfer execution, knowledge, or speech authority to that component.

- `dialogue_manager` owns the dialogue turn lifecycle and user-facing speech realization.
- `chatbot_llm` owns user-facing semantic interpretation: intent derivation, route selection (dialogue or execution) and goal text for planner.
- `nao_orchestrator` owns deterministic planner admission and request normalisation. It also owns executable-step validation, dispatch, step status, and feedback publication.
- `planner_llm` owns structured skill-plan generation, feedback-driven goal-state handling, replanning decisions, and dialogue-act intent.
- Skills own live interaction with the robot, scene, and KB evidence required to complete their operation truthfully.

4.2 Runtime Contract Overview

Table 4.1 summarises each runtime contract, its interface or payload, and its owner.

Table 4.1: Runtime contract map and ownership in the active dialogue, planning, and execution stack.

ID	Contract	Interface / payload	Ownership
1	Chatbot turn output	Internal chatbot result before ROS planner publication.	<code>dialogue_manager</code> invokes the chatbot path; <code>chatbot_llm</code> produces the semantic result.
2	Planner request and admission	Candidate topic <code>/nao_orchestrator/planner_request</code> . Admitted topic <code>/planner/request</code> . Planner-facing payload is carried in <code>Intent.data</code> .	<code>chatbot_llm</code> proposes the execution request; <code>nao_orchestrator</code> admits and normalizes it; <code>planner_llm</code> consumes the admitted request.
3	Compact grounding	<code>grounded_context</code> inside Contract 2.	Scene and KB projection provide compact symbolic context for <code>chatbot_llm</code> and <code>planner_llm</code> .
4	Shared skill entry	Registry projection consumed by the planner and orchestrator; its adapter mapping identifies the receiving skill server.	<code>skill_common</code> defines the shared capability shape; <code>planner_llm</code> uses it for prompt construction and <code>nao_orchestrator</code> uses it for plan validation and dispatch.
5	Planner output	Topic <code>/intents</code> . Structured plan payload in <code>Intent.data.plan</code> .	<code>planner_llm</code> produces the plan; <code>nao_orchestrator</code> validates and dispatches executable steps.
6	Skill execution result	ROS action result or service response with a typed skill-result payload.	<code>nao_orchestrator</code> dispatches the call; the skill server owns effects and fresh evidence; verified KB effects cross <code>kb_skills</code> .
7	Execution feedback	Topic <code>/planner/execution_feedback</code> . Structured JSON feedback payload.	<code>nao_orchestrator</code> publishes feedback; a goal-keyed planning-state helper inside <code>planner_llm</code> uses it for replanning, clarification, or failure handling.
8	Planner dialogue act	Topic <code>/planner/dialogue_act</code> . Structured communication request.	<code>planner_llm</code> emits dialogue-act intent; <code>nao_orchestrator</code> relays it; <code>dialogue_manager</code> realizes the final speech.
9	RDF knowledge state and raw evidence	KnowledgeCore statements through <code>/kb/query</code> and <code>/kb/revise</code> ; source evidence includes tracked-person state, <code>/scene/summary</code> , and skill results.	Source nodes own fact freshness; <code>kb_skills</code> owns KB transport; compact projections and feedback preserve provenance without replacing raw evidence.

The overview keeps only the fields needed to identify each contract. Later sections add the runtime metadata needed for orchestration, feedback-driven planning, tracing, and debugging.

4.3 Contract 1: Chatbot Turn Output

The intent in this contract is the user’s goal as extracted by the chatbot. It is not a planner output.

```

1 {
2   "verbal_ack": "Okay, I will do that.",
3   "route": "execution",
4   "confidence": 0.82,
5   "user_intent": {
6     "type": "look_at",
7     "goal_text": "look at the person",

```

```

8   "scene_targets": ["person"],
9   "request_kind": "new_goal"
10  }
11 }

```

Table 4.2 identifies the fields that remain visible after the chatbot completes one semantic routing decision.

Table 4.2: Fields in Contract 1: chatbot turn output.

Field	Meaning
<code>verbal_ack</code>	Optional immediate acknowledgement generated by the chatbot layer. This may be spoken or logged before execution begins, depending on the dialogue policy.
<code>route</code>	Turn classification produced by <code>chatbot_llm</code> . Supported values include <code>dialogue</code> , <code>knowledge_query</code> , and <code>execution</code> . Only <code>execution</code> turns enter the planner path; <code>dialogue</code> and <code>knowledge_query</code> remain non-execution turns but are still useful for tracing.
<code>confidence</code>	Confidence assigned by the chatbot or intent-extraction stage. It is useful for debugging and possible admission policy, but it is not a planner priority score.
<code>user_intent.type</code>	Normalized user-intent label extracted by <code>chatbot_llm</code> , such as <code>look_at</code> , <code>find_object</code> , <code>scan</code> , or <code>navigate_to</code> .
<code>user_intent.goal_text</code>	Natural-language objective passed toward the planner path as the task to satisfy. This field preserves the user's goal in a compact form.
<code>user_intent.scene_targets</code>	Optional target hints extracted from the utterance, such as <code>person</code> , <code>cup</code> , or <code>table</code> . These are hints, not validated grounded entities.
<code>user_intent.request_kind</code>	Proposed goal transition class, such as <code>new_goal</code> , <code>update_goal</code> , <code>answer_to_clarification</code> , or <code>cancel_goal</code> . The planner gate inside <code>nao_orchestrator</code> should normalize and admit this field before publishing <code>/planner/request</code> .

4.4 Contract 2: Planner Request

The planner request is the task description generated by the orchestrator from the chatbot output and consumed by the planner.

```

1 {
2   "goal_id": "goal_turn_123",
3   "request_kind": "new_goal",
4   "goal_text": "look at the person and report completion",
5   "normalized_intents": ["look_at"],
6   "grounded_context": {
7     "entities": [
8       {
9         "id": "anonymous_person_ehfbf",
10        "label": null,
11        "kind": "person",
12        "class": "Human",
13        "visible": true,
14        "relations": []
15      }
16    ]
17  }
18 }
```

Table 4.3 separates request identity, goal semantics, and grounded evidence before the planner is invoked.

Table 4.3: Fields in Contract 2: minimal planner-facing request.

Field	Meaning
<code>goal_id</code>	Identifier for the active objective. It groups replans and feedback for the same task.
<code>request_kind</code>	States whether this is a new goal, an update to the current goal, a cancellation, or another supported transition. <code>chatbot_llm</code> may propose it, but <code>nao_orchestrator</code> normalizes and admits it before publishing <code>/planner/request</code> .
<code>goal_text</code>	Objective extracted from the user utterance. This is the central planning input for <code>planner_llm</code> .
<code>normalized_intents</code>	Intent labels extracted by <code>chatbot_llm</code> . They help <code>planner_llm</code> choose suitable skills from the available skill registry.
<code>grounded_context</code>	Compact scene graph used by the planner as the current symbolic state of the world. It contains validated entities and relations rather than raw detector output.

Implementation metadata can still exist on the ROS envelope or runtime trace. Examples include request identifiers, dialogue turn identifiers, planner mode, parent goal ID, or super-

seded goal ID. These fields are useful for tracing, debugging, and goal management, but they are not the core task description consumed by `planner_11m`.

4.5 Contract 3: Grounded Context

The planner receives a compact, JSON-native view of relevant world state. This is best described as a compact scene graph derived from the KB, continually updated by the perception pipeline (human/person detection and object perception). Raw KB triples, detector metadata, and tracker-derived details remain available through scene/KB evidence paths to skills, but they are not duplicated into every LLM prompt.

```

1 {
2   "grounded_context": {
3     "entities": [
4       {
5         "id": "cup_jrjic",
6         "label": "cup",
7         "kind": "object",
8         "class": "Cup",
9         "visible": true,
10        "relations": [
11          {"predicate": "dbp:color", "object": "blue"},
12          {"predicate": "oro:isOn", "object": "table_1"}
13        ]
14      },
15      {
16        "id": "anonymous_person_ehfbf",
17        "label": null,
18        "kind": "person",
19        "class": "Human",
20        "visible": true,
21        "relations": []
22      }
23    ]
24  }
25 }
```

Table 4.4 defines the compact entity and relation fields used by dialogue and planning.

Table 4.4: Fields in Contract 3: grounded context.

Field	Meaning
<code>id</code>	Stable entity handle used by planner steps and skill arguments. This is the preferred reference when a skill needs to target a known person, object, or location.
<code>label</code>	Human-readable name or class hint. It may be empty for anonymous people or entities that should not be assigned a personal name.
<code>kind</code>	Entity category, such as <code>object</code> , <code>person</code> , or another supported runtime category. This prevents people from being treated as generic objects.
<code>class</code>	Compact semantic class used by the planner for reasoning, such as <code>Cup</code> , <code>Human</code> , or <code>Table</code> .
<code>visible</code>	Indicates whether the entity is currently grounded in the latest accepted scene or KB state. It should not be treated as a guarantee that the entity will still be available at execution time.
<code>relations</code>	Bounded list of relevant semantic relations, such as colour, containment, support, proximity, or ownership-style facts. These relations add task-relevant context without exposing the full KB.

The compact view should omit detector score, image coordinates, backend, source, and recency by default. Those values are execution evidence, not planning semantics, unless a future skill explicitly needs them in its own action arguments.

4.6 Contract 4: Shared Skill Entry

The planner and orchestrator use one shared entry shape to describe a skill before it is placed in a prompt, admitted for execution, or mapped to an action server. This entry is a capability contract, not a claim that every listed adapter is available in every launch profile. It separates the information needed to select a skill from the information needed to verify its result.

```

1 {
2   "object_id": "navigate_to",
3   "kind": "skill",
4   "category": "navigation",
5   "aliases": ["go_to", "move_to_location"],
6   "params": ["target", "location"],
7   "required_params": ["target"],
8   "preconditions": ["navigation is enabled",
9                     "target is known or resolvable"],
10  "expected_effects": ["robot arrives at the target location"],
11  "observable_success": ["status"],
12  "failure_modes": ["path_blocked", "unknown_location",
13                  "safety_disabled"],
14  "planner_guidance": ["use for a semantic destination"],

```

```

15  "result_schema": {"required": ["skill", "status", "summary_text"]},
16  "safety_flags": ["navigation"],
17  "implementation_status": "fake",
18  "robot_adapter_mapping": "fake_skills.navigate_to",
19  "supports_failure_injection": true,
20  "metadata": {"runtime_callable": true}
21 }

```

Table 4.5 names the common capability fields consumed by prompt construction, admission, dispatch, and result interpretation.

Table 4.5: Shared skill-entry fields used by planning and execution.

Field	Runtime role
<code>object_id</code> , <code>kind</code> , <code>category</code> , <code>aliases</code>	Canonical planner name, semantic group, and accepted compatibility names.
<code>params</code> , <code>required_params</code>	Typed planner-facing arguments and the subset required before admission.
<code>preconditions</code>	Grounding or state assumptions that must be satisfied or explicitly owned by a scenario. They do not prove a future effect.
<code>expected_effects</code>	Intended state changes or evidence products used to select and order skills.
<code>observable_success</code>	Result or trace evidence that can support completion after execution.
<code>failure_modes</code>	Typed outcomes used by the orchestrator and the goal-state helper to stop, clarify, or request replanning.
<code>planner_guidance</code>	Bounded selection and ordering guidance inserted into the planner prompt.
<code>result_schema</code>	Required and optional result fields, including the summary and evidence payload.
<code>safety_flags</code>	Safety-relevant labels used during admission and dispatch.
<code>metadata.runtime_callable</code>	Whether the registry permits planner-visible runtime use.
<code>implementation_status</code>	Current implementation class, such as first-party, fake, or hybrid.
<code>robot_adapter_mapping</code>	Adapter or skill server that receives the dispatched call.
<code>supports_failure_injection</code>	Whether controlled failure outcomes are available for validation. Launch-profile checks still establish whether the selected endpoint is active.

The same shape is projected to the planner prompt, deterministic orchestrator checks, fake-skill policies, and adapter configuration. Preconditions constrain admission, while expected effects remain hypotheses until a skill returns observable evidence. This distinction is used by `report_result`: a final summary must be derived from result payloads or trace evidence rather than copied from a planner-generated claim. The complete runtime inventory is provided in Appendix A.1.

4.7 Grounded Context Pipeline

Contract 3 is produced by a multi-source projection rather than a direct detector-to-prompt copy. Figure 4.1 shows how camera and simulator observations, tracked people, and KnowledgeCore statements become bounded context for the chatbot and planner while raw evidence remains at its owning interface.

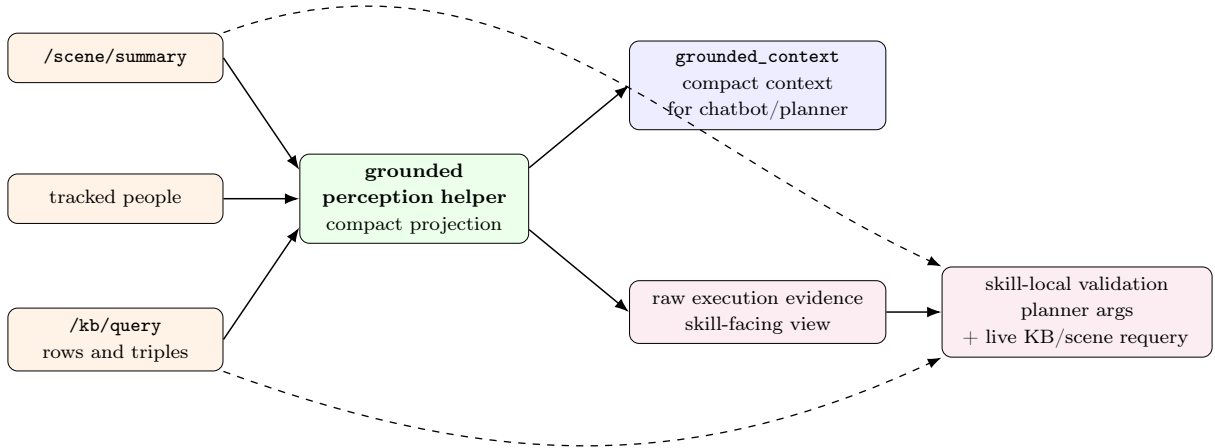


Figure 4.1: Grounded-context pipeline. Scene and KB evidence are projected by a grounded perception helper into a compact `grounded_context` for `chatbot_llm` and `planner_llm`. Skills receive planner-approved arguments through execution, while also retaining access to raw scene and KB evidence for live requery and validation before reporting outcomes.

Derivation rules:

1. Use text-derived entities only when unable to confirm entities with KB.
2. Keep anonymous people anonymous; do not invent personal names.
3. Keep IDs and labels separate.
4. Bound relation count per entity to keep prompt size manageable.

4.8 Planner LLM Internals

After the planner request is admitted, `planner_llm` builds a prompt from the compact request, grounded context, the shared skill-entry projection, allowed step types, and the output contract. The prompt carries the goal and request kind, normalized intent hints, relevant grounded entities, permitted skill names, argument requirements, and recovery vocabulary. A bounded feedback payload is added when the current goal is responding to an execution event. Rule-based shortcuts may produce deterministic plans for simple motions, but the normal path remains model-generated and contract-checked.

The provider response is handled as an untrusted candidate. The planner extracts the first

admissible JSON object, normalizes plan and step metadata, validates skill names, arguments, ordering, and failure policies, then performs one bounded validation retry when the prompt pack enables it. The retry receives the validation error and a bounded portion of the previous output. A second invalid result becomes a typed planner failure or clarification outcome. No raw model text is sent to `/intents`, and no invalid plan is passed to the orchestrator.

The planner prompt should not contain raw detector coordinates, full RDF dumps, or stale perceptual reports. Those belong to skills and execution evidence.

4.9 Contract 5: Planner Output

The planner output is a plan: a sequence of skill steps to achieve the user goal. It should focus on executable content and minimal lineage. Communication policy and retry metadata are still valid runtime fields when used by the stack, but this controlled version foregrounds the ordered `steps` list because that is the essential planner output.

```

1 {
2   "plan": {
3     "goal_id": "goal_turn_123",
4     "plan_id": "plan_123",
5     "plan_version": 1,
6     "status": "planned",
7     "steps": [
8       {
9         "id": "step_1",
10        "type": "skill",
11        "name": "look_at",
12        "args": {"target_frame": "anonymous_person_ehfbf"},
13        "requires": [],
14        "on_failure": "replan"
15      }
16    ]
17  }
18 }
```

Table 4.6 identifies the plan and step fields that must survive deterministic validation before dispatch.

Table 4.6: Fields in Contract 5: planner output.

Field	Meaning
<code>goal_id</code>	User objective that this plan attempts to satisfy. It links the planner output to the active goal.
<code>plan_id</code>	Identifier for this concrete plan proposal. It is useful when the runtime keeps plan history or compares revised plans for the same goal.
<code>plan_version</code>	Revision number for replans of the same goal. This helps distinguish the initial plan from later feedback-driven planning outputs.
<code>status</code>	Planning outcome for this plan, such as <code>planned</code> , <code>needs_clarification</code> , or <code>failed_to_plan</code> . This describes whether <code>planner_llm</code> produced an executable plan, not whether the robot has already executed it.
<code>steps</code>	Ordered list of skill calls produced by the planner. This is the essential planner output consumed by <code>nao_orchestrator</code> .
<code>steps[].id</code>	Stable step identifier used for execution feedback, dependency tracking, and possible replan joins.
<code>steps[].name</code>	Skill name selected from the allowed skill registry. The planner should not invent skills or use robot-specific APIs directly.
<code>steps[].args</code>	Skill arguments. They should refer to known entity IDs, explicit primitive values, or arguments supported by the selected skill schema.
<code>steps[].requires</code>	Dependencies between steps, if any. This allows the runtime to know which steps depend on previous successful outcomes.
<code>steps[].on_failure</code>	Recovery policy proposed by <code>planner_llm</code> for this step. If the step fails, <code>nao_orchestrator</code> may fail safely, continue if allowed, request user input, or send feedback back to <code>planner_llm</code> for replanning. Allowed values include <code>replan</code> , <code>clarify</code> , <code>ask_user</code> , <code>continue</code> , <code>ignore</code> , and <code>fail</code> .

Runtime recovery metadata. The current contract supports recovery metadata as skill failure may require controlled recovery. Each step may carry a `retry_budget`, and the plan envelope may also carry a plan-level `retry_budget`. This does not mean that `retry` is a separate `on_failure` value. Instead, retry behaviour is controlled by `nao_orchestrator` using the available retry budget together with the step’s `on_failure` policy. The plan envelope may also carry `communication_policy`. This controls when the planner/dialogue path should communicate progress, completion, clarification needs, or failure.

When a step fails with `on_failure = replan`, `nao_orchestrator` halts or blocks continuation, publishes execution feedback to `planner_llm`, and waits for a revised validated plan before dispatching further actions. This keeps execution authority inside `nao_orchestrator`, while allowing recovery to be driven by feedback-driven planning rather than by hard-coded local replanning.

These fields are included because they support runtime recovery, traceability, and human-robot interaction, but the essential planner output remains the ordered `steps` list. If the planner cannot produce a plan, the output should represent that as a planning result rather than filling

the plan with ambiguous failure fields. For the simple version, **steps** is the most important field.

4.10 Contract 6: Skill Execution Result

Every dispatched skill returns a typed result before the orchestrator can mark its step as completed. The result distinguishes the execution status from the evidence that supports it. For state-changing skills, the evidence may also contain explicit KB effects. These effects are proposed by the skill that observed or produced the change; they are not inferred from the planner's expected-effects text in Contract 4.

```

1 {
2   "skill": "place_object",
3   "status": "succeeded",
4   "summary_text": "I placed cup_1 on table_1.",
5   "evidence": {
6     "object_id": "cup_1",
7     "destination_id": "table_1",
8     "kb_effects": [
9       {"operation": "remove", "statement": "robot oro:holds cup_1"},
10      {"operation": "add", "statement": "cup_1 oro:isOn table_1"}
11    ]
12  },
13  "failure": null,
14  "metadata": {"fake": true}
15 }
```

The orchestrator applies `evidence.kb_effects` only after a successful result and routes the statements through `kb_skills`. Removal effects clear stale holding or location relations before replacement facts are added. When the query seam is available, the orchestrator verifies that removed statements no longer resolve and that added or revised statements do. A failed mutation or post-condition check converts the step into a failure and is propagated through Contract 7; later dialogue therefore cannot treat an unverified material change as current KB truth. Real and fake skills use the same result shape, although the physical evidence available to them differs.

4.11 Contract 7: Execution Feedback

For the simple version, execution feedback can be minimal. The planner needs to know which goal is affected, which step failed or succeeded, and a compact result if that result is useful for replanning or dialogue. A larger payload is justified only when the goal-state helper, orchestrator, traces, or debug tooling use the extra fields.

```
1 {
2   "goal_id": "goal_turn_123",
3   "status": "failed",
4   "step": {
5     "id": "step_1",
6     "name": "look_at"
7   },
8   "reason": "target frame unavailable",
9   "result_payload": {
10    "target_found": false,
11    "target_kind": "person"
12  }
13 }
```

Runtime feedback may include additional fields used by `nao_orchestrator`, logging, and the goal-state helper. Lineage uses `plan_id` and `plan_version`; event handling may add `event_type`, `validation_status`, `blocking`, and `needs_user_input`; observability may add `timestamp_sec`, `result_summary`, and retry metadata. These fields support execution control and debugging, while the minimal planner-facing feedback remains the goal, step, status, reason, and result payload.

4.12 Contract 8: Planner Dialogue Act

Planner dialogue acts represent planner intent to communicate, not direct speech execution. They are relayed through the orchestrator and realized by the dialogue layer.

```
1 {
2   "goal_id": "goal_turn_123",
3   "act": "ask_clarification",
4   "await_user_response": true,
5   "reason": "multiple visible people",
6   "text_hint": "Which person should I look at?",
7   "slots_needed": ["target_person"]
8 }
```

Allowed dialogue acts are acknowledgement, progress update, clarification, help request, failure explanation, completion notification, and cancellation notification. The planner does not speak directly; it emits a structured communication request.

4.13 Contract 9: RDF Knowledge State and Raw Evidence

KnowledgeCore represents symbolic world state as an RDF-style graph. Each statement has a subject, predicate, and object. Stable subjects identify people, objects, places, supports, and the robot; predicates express class membership, attributes, visibility, containment, and spatial relations. The same statement shape is used by preloaded fixtures, detector-derived object facts, explicit user-authorized KB mutations, and verified post-skill effects. This allows a query pattern to match facts independently of which approved source produced them.

```

1 codex_lab_cup rdf:type Cup
2 codex_lab_cup dbp:color red
3 codex_lab_cup oro:isOn codex_lab_table
4 codex_lab_room oro:contains codex_lab_table
5 person_alex rdf:type Human
6 myself sees person_alex

```

The principal predicate families are shown in Table 4.7. A class statement such as `rdf:type Human` also preserves the semantic distinction between a person and an object even when both can be action targets.

Table 4.7: Principal RDF-style predicate families used by the runtime KB.

Role	Representative predicates	Runtime meaning
Entity class	<code>rdf:type</code>	Distinguishes people, objects, supports, rooms, and other planner-relevant classes.
Attributes	<code>dbp:name</code> , <code>dbp:color</code>	Provides stable semantic labels and disambiguating properties.
Observation	<code>sees</code> , <code>visual-centre</code> , <code>score</code> , <code>source</code> , and last-seen predicates	Records who observed an entity and the bounded provenance/freshness of a detector observation.
Spatial relations	<code>oro:isOn</code> , <code>oro:isAt</code> , <code>oro:isIn</code> , <code>oro:contains</code>	Represents support, destination, room membership, and containment used by grounding and skill effects.

KB mutations use `add`, `revise`, and `remove` operations on this shared statement form through `/kb/revise`; reads use query patterns through `/kb/query`. Transient perception facts carry a finite lifespan, while explicit fixture and verified skill-effect facts follow the lifetime declared by their owner. `hri_face_detect_yunet` and `hri_person_manager` publish person identities and frame state through their public tracked-person interfaces. In the integrated simulator, corresponding person knowledge can be materialised in the same graph without assigning person tracking to `nao_scene_grounding`. Object detections remain owned by `nao_scene_grounding`, which publishes `/scene/summary` and detector-derived KB statements.

Raw evidence remains an ownership boundary rather than one universal message. Tracked-person topics, detector observations, KnowledgeCore query rows, robot or simulated action results, and structured trace events stay at their owning interfaces. The internal knowledge snapshot is a bounded textual serialization of selected query rows; `grounded_context` in Contract 3 is the compact entity-and-relation projection consumed by `chatbot_llm` and `planner_llm`. Contract 6 skill results and Contract 7 planner feedback are separate projections for execution and recovery. None replaces the underlying source evidence.

Chapter 5

Implementation: ROS 2 Nodes and APIs

This chapter describes how the architecture introduced in Chapter 3 and the contracts defined in Chapter 4 are realised in the ROS 2 workspace. The implementation deliberately separates semantic interpretation, planning, deterministic execution, perception grounding, symbolic knowledge, and skills. LLM components can therefore interpret or propose actions, but physical effects remain mediated by typed ROS interfaces and validated skill contracts.

The workspace combines locally developed integration packages with upstream-aligned HRI components. In particular, `dialogue_manager` provides a bridge between robot and user interaction, while the `chatbot_llm` package supplies intent extraction, response generation, and the turn-processing interface extended for this system. The planner-facing contracts, orchestrator, perception pipeline, fake-skill infrastructure, skill registry, and NAO adapters are integrated in the `nao-ros4hri-bridge` workspace. The contribution is therefore the complete interaction and execution integration rather than a replacement dialogue package or an isolated planner.

5.1 Runtime Flow

A user utterance first enters `dialogue_manager`, which sends a `DialogueInteraction` request to `chatbot_llm` with the dialogue state required for the turn. The chatbot combines this state with a symbolic representation of the current scene derived from the knowledge base, performs the call to the LLM for intent classification and response generation, and classifies the result as `dialogue`, `knowledge_query`, or `execution`. `dialogue` and `knowledge_query` turns return a response to the dialogue owner without entering the planner. `execution` turns additionally produce a structured planner request with goal metadata, normalised intent hints, dialogue history and current world state. This request is published to `/nao_orchestrator/planner_request` for admission.

Candidate requests are published to the planner-admission endpoint defined by Contract 2 (Table 4.3). This endpoint is owned by `nao_orchestrator`, which normalises the payload, establishes goal lineage, and forwards only admissible requests on `/planner/request`.

`planner_llm` generates a structured plan from that request and the shared skill registry, including execution order, relevant KB entities such as objects, locations, and people, and the metadata required for deterministic dispatch. The resulting planner output returns to the orchestrator, where its schema, skill names, arguments, and step ordering are checked before any action or service is invoked. Step outcomes are published on `/planner/execution_feedback`. A goal-keyed planning-state helper inside `planner_llm` consumes this feedback when the orchestrator reports a relevant event, such as a failed step, a clarification requirement, or a recoverable condition. It updates the active planning state and may request a replan or planner dialogue act. The planner node itself does not monitor or control robot execution.

This flow enforces three runtime constraints from the requirements in Table 3.1. First, a dialogue-only turn cannot reach robot execution unless the chatbot selects the execution route and the orchestrator admits the resulting request (FR2–FR3). Second, planner output remains a candidate plan until its schema, skill names, arguments, failure policies, and lineage fields pass deterministic validation (FR5). Third, KB facts supplied with a planner request describe what the system knows when planning begins. They can support target selection, but they cannot prove that a later action succeeded or that the world remained unchanged; the responsible skill must provide fresh execution evidence before completion is reported.

Figure 5.1 shows how the nodes, Python packages, ROS interfaces, external backends, validation inputs, and future extensions fit together.

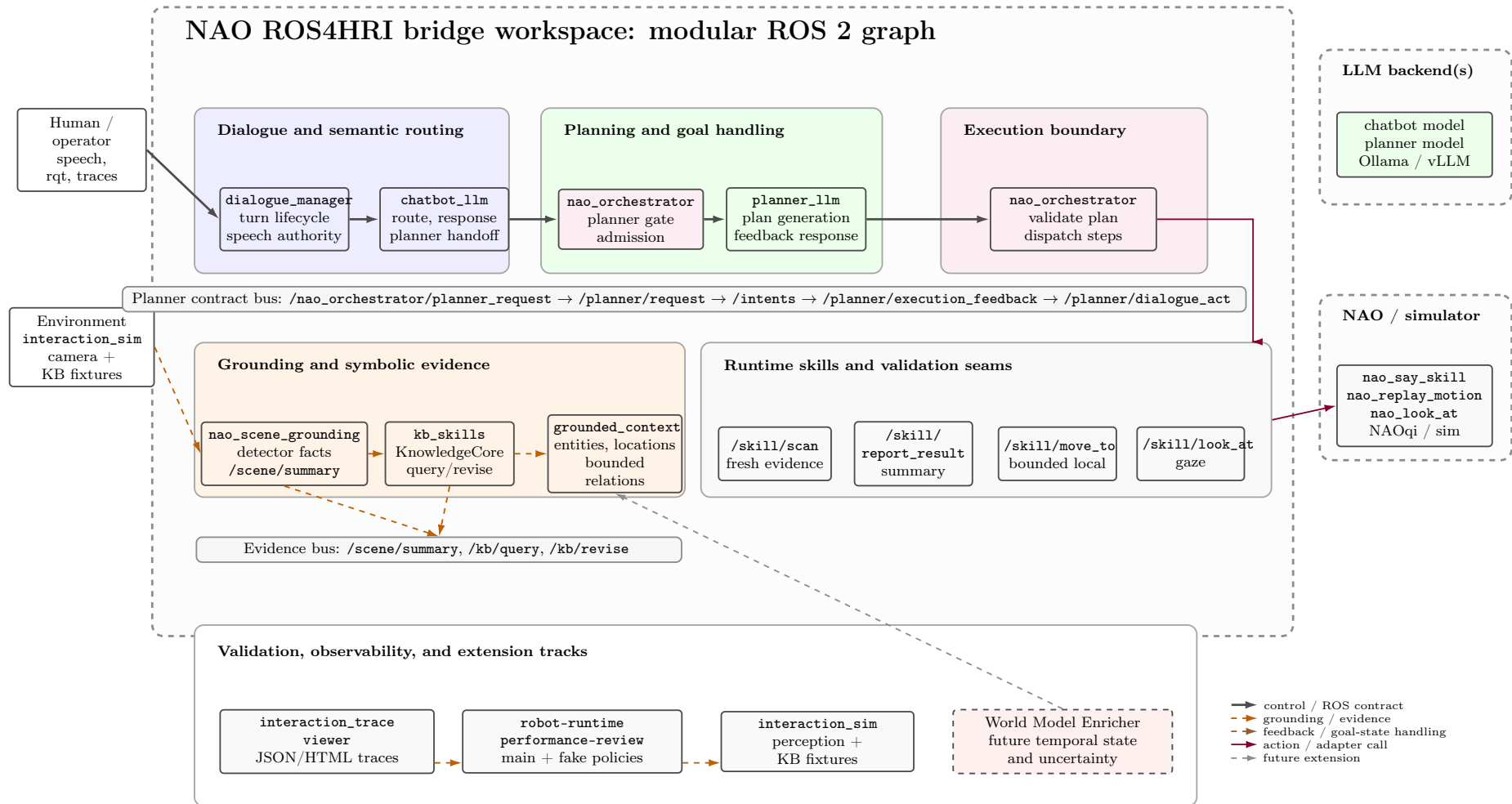


Figure 5.1: Full-stack implementation of the modular NAO interaction and execution system. Dialogue components are blue, grounding and evidence components orange, planning components green, execution components purple, and skills or adapters neutral grey. The `interaction_sim` and KB-fixture paths provide controlled validation inputs. The dashed red World Model Enricher box denotes future work rather than an implemented runtime component.

5.2 ROS Nodes

5.2.1 `dialogue_manager`

`dialogue_manager` is the ROS 2 lifecycle component responsible for multi-modal communication between the robot and its interlocutors. Following the upstream package design¹, it tracks concurrent conversations per person and per interaction group, preserves chronological utterance histories, and associates incoming `hri_msgs/LiveSpeech` streams with the relevant dialogue. Group utterances are fanned out to the corresponding personal and group histories, while prior summaries can be restored when a known interlocutor begins a new session. The node invokes the chatbot through `DialogueInteraction`, exposes the high-level `/skill/chat`, `/skill/ask`, and `/skill/say` interfaces, and realises final user-facing expressions through the configured Say sub-skill. The planner integration therefore extends an existing multi-user dialogue lifecycle rather than replacing it.

This ownership boundary is also a runtime constraint: planning and execution packages may publish structured communication requests, but they do not call the speech backend directly merely because they have produced status information. This preserves the separation between dialogue and execution introduced by FR2 and the ownership map in Table 4.1.

Planner dialogue acts return through the same ownership boundary. `planner_llm` publishes structured acknowledgement, clarification, failure, or completion acts on `/planner/dialogue_act`; `nao_orchestrator` relays the admitted act on `/nao_orchestrator/planner_dialogue_act`; and the dialogue layer chooses the final wording and speech mechanism. This implementation of Contract 8 preserves one utterance authority per semantic event and makes duplicate acknowledgement or completion speech observable in the validation traces.

5.2.2 `chatbot_llm`

`chatbot_llm` is a lifecycle-managed semantic routing node built on the existing chatbot scaffold. Its turn callback receives a `DialogueInteraction` and the conversation history assembled by `dialogue_manager`. Before response generation or any planner handoff, it queries the KB through `kb_skills`, reads the most recent `/scene/summary`, and passes both sources through the `PlannerHandoff` projection helper. The result is the compact `grounded_context` defined by Contract 3 (Table 4.4 and Figure 4.1). It contains a bounded set of entities, locations, and task-relevant relations instead of raw detector streams or unrestricted KB rows. This is the principal KB-to-chatbot seam: the same projection informs dialogue and knowledge answers on every user turn, and is attached to Contract 2 only when the selected route produces a planner request.

The turn engine implements the route field from Contract 1 as follows:

- A `dialogue` route covers conversational and task-discussion turns and does not publish planner work.

¹Package source: https://github.com/ieverythng/dialogue_manager.

- A `knowledge_query` route answers from current symbolic evidence without causing a physical action. For example, “what can you see?” can use the current grounded context, whereas “scan and tell me what you see” requests fresh sensing and is eligible for execution.
- An `execution` route constructs planner-facing metadata such as goal text, request kind, normalised intent hints, and target KB entities. It publishes a candidate request to `/nao_orchestrator/planner_request`, but it does not create a skill sequence.

The final point implements FR3 and the admission ownership in Contract 2: `chatbot_llm` may propose execution, but only `nao_orchestrator` can admit the request and only a validated planner output can reach skill dispatch.

The canonical behavioural policy resides in `src/chatbot_llm/config/chat_prompt_pack.yaml`. Python builders provide structural framing and runtime context, while the chatbot sequence uses response and intent stages that share the same system prompt, route policy, skill catalogue, and grounded context. The baseline is `response_first`. In planner mode, the response-stage call returns the natural `verbal_ack` together with the route and confidence fields from Contract 1. The intent stage then defines the user intents and, for an execution route, constructs the candidate planner handoff from that staged result. The response stage therefore performs route selection as part of the same semantic decision that produces the acknowledgement; it does not create an executable plan. In the `intent_first` ablation, the intent stage runs first, its route is checked and locked, and the response stage generates wording under that locked route. The comparison tests whether early route commitment changes contradictions between the spoken response and planner admission. The proposed Intent–Route–Response (IRR) variant goes further by producing the three outputs atomically, but it remains a future or separately scored implementation until its trace and route-consistency behaviour are validated.

Every completed turn emits `/chatbot_llm/turn_trace`. The trace records the final route, intent source and confidence, grounded-context visibility, spoken acknowledgement, and whether a planner handoff was published. It therefore provides the observable link between Contract 1, Contract 2 admission, and the route-consistency evaluation in Chapter 6.

Figure 5.2 shows the turn stages, grounded-context input, route decision, user-facing response, trace output, and planner handoff.

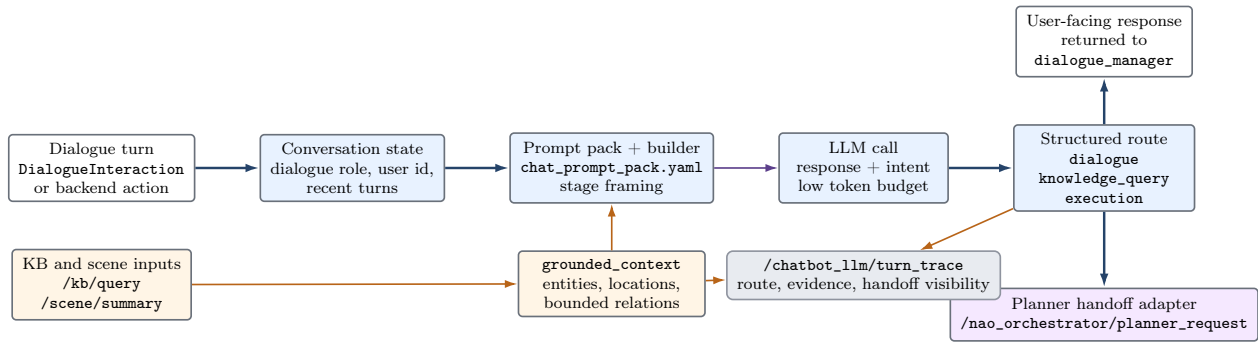


Figure 5.2: Internal `chatbot_llm` path. The node owns semantic interpretation and candidate planner metadata, but not plan construction, dispatch, or speech execution.

5.2.3 `planner_llm`

`planner_llm` is the high-level planning node and contains a goal-keyed planning-state helper for feedback-driven state transitions. The helper is a subprocess-like part of the node, not an independent node, and has no execution authority. The node subscribes to admitted `/planner/request` messages rather than directly consuming chatbot output. Through `planner_common`, each request is normalised into the canonical contract with goal metadata nested under `plan`. The planner prompt combines the goal text, request kind, normalized intent hints, grounded context, the shared skill-entry projection from Contract 4, permitted step types, and the required JSON output envelope. Planner policy is maintained in `src/planner_llm/config/planner_prompt_pack.yaml`; the runtime builder supplies request-specific structure without duplicating that policy in Python source.

The planner prompt is assembled from the admitted request, bounded grounded entities, the skill registry fields relevant to selection and ordering, the permitted step types, and the planner output contract. The model is told which skills are available, which arguments are required, which effects are expected, which result evidence can support completion, and which failure modes may require clarification or replanning. This registry projection is the main link between the model’s language-level proposal and the typed ROS execution surface.

The model response is not treated as executable until the planner engine extracts the candidate object, normalises its schema, validates required metadata and step fields, and applies one bounded repair retry when configured. Unsupported skill names, malformed arguments, invalid recovery policies, or unparseable output become typed failure or clarification outcomes. The planner records the selected output mode, including a normal model plan or one of the explicitly named deterministic fallback modes described in Section 5.3. Valid plans are published on `/intents` for independent orchestrator validation.

Evidence-producing plans receive an additional reporting check. When `report_result` follows a live sensing, navigation, motion, or manipulation skill, `planner_common` requires the reporting step to omit planner-authored result wording. Its bounded compatibility normalisation removes fields such as `summary_text`, `result_summary`, `text`, `message`, and `utterance` before the plan is validated again. The empty reporting step is later resolved

from the preceding Contract 6 result payload. This prevents a syntactically valid plan from presenting an expected effect as an observed outcome.

The node also subscribes to `/planner/execution_feedback`. The embedded planning-state helper associates relevant events with the active goal and asks the planning path to respond when the orchestrator reports completion, failure, a recoverable condition, or a need for user input. That response may be a revised plan or a structured dialogue act; it is not robot control. The planner never invokes a robot adapter and never speaks directly. Its user-facing output must cross the orchestrator relay and dialogue owner.

Figure 5.3 shows the planner model call together with its validation, retry, feedback, and publication paths.

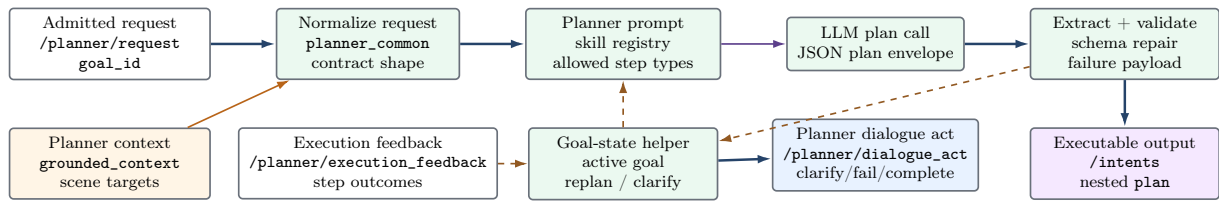


Figure 5.3: Internal `planner_llm` path, from admitted request through validated plan envelope and feedback-driven goal-state handling.

5.2.4 Goal-Keyed Planning-State Helper

The goal-keyed planning-state helper maintains planning state under an active `goal_id`; it does not monitor or control robot execution. Each accepted plan additionally carries a `plan_id` and monotonically meaningful `plan_version`; stable `step_id` values allow a newer version to preserve already completed work when a replan joins the same goal. Feedback from an older or unrelated version is rejected rather than being allowed to complete the current goal accidentally.

The helper distinguishes planning, replanning, waiting-for-user, completed, and failed states. Step feedback may move the goal forward, expose a recoverable condition, or produce a structured dialogue act. If a replacement plan is necessary, `planner_llm` creates and validates it under a newer version. `nao_orchestrator` never invents a local substitute plan, which preserves a single planning authority while keeping execution deterministic. The helper maintains order and lineage across these transitions; it does not supervise the robot or replace the orchestrator’s execution checks.

5.2.5 Skill Registry (`skill_common`)

The planner-facing capability model is derived from the canonical skill registry maintained by `skill_common`. Contract 4 and Table 4.5 define the shared entry shape: canonical name and aliases, typed parameters, preconditions, expected effects, observable success evidence, failure modes, result schema, safety flags, runtime availability, and executor mapping. The

planner receives a bounded projection of these fields when it selects and orders skills. The orchestrator uses the same names, argument requirements, and availability information when it validates Contract 5 plans and dispatches Contract 6 calls. This arrangement provides a machine-checkable guard against prompt and executor drift. The final scored source revision is accepted only when the generated planner and documentation projections pass the registry consistency checks against the canonical entries.

The registry also separates semantic availability from the adapter selected by a launch profile. A skill may be implemented by a first-party robot action, a deterministic fake server, or both. The profile must still prove that the selected endpoint is running before execution. Table 5.1 lists the complete planner-visible skill set in compact form; Appendix A.1 gives the argument, effect, failure, and executor fields for each skill.

Table 5.1: Planner-visible runtime skills and their current execution profiles.

Skill(s)	Contract role	Current adapter/profile	Fake validation
<code>scan</code>	Refresh scene evidence and return a typed scan payload.	First-party <code>/skill/scan</code> server.	No separate fake route.
<code>find_object</code> , <code>inspect_area</code>	Return object, person, or area evidence with typed ambiguity and backend failures.	Deterministic fake server in the evaluated profile.	Yes.
<code>navigate_to</code>	Reach a semantic destination such as a person, object, or location.	Deterministic fake navigation endpoint.	Yes.
<code>walk_to</code>	Execute a short body-relative locomotion segment.	Real <code>/skill/move_to</code> or fake endpoint, selected by profile.	Yes.
<code>pick_object</code> , <code>place_object</code> , <code>bring_object</code>	Change object holding, support, and delivery relations and return KB effects.	Deterministic manipulation servers in the current evaluated stack.	Yes.
<code>perform_motion</code> , <code>look_at</code>	Change posture, head orientation, or gaze target.	First-party replay, head-motion, and look-at adapters; fake endpoints are selectable in simulation.	Yes.
<code>wave_greet</code>	Perform a guarded social greeting gesture.	Real replay-motion gesture or fake endpoint.	Yes.
<code>kb_add</code> , <code>kb_revise</code> , <code>kb_remove</code>	Apply explicit symbolic mutations through KnowledgeCore.	First-party orchestrator to <code>kb_skills</code> path.	No hardware fake required.
<code>ask_user</code> , <code>report_result</code>	Request missing information or report evidence-derived execution results.	First-party dialogue relay and report/say skill servers.	Trace-driven.

5.2.6 nao_orchestrator and the Execution Boundary

`nao_orchestrator` is the deterministic execution layer between an admitted intent and a robot or simulated effect. It is the common deployment manager for registered skills: it resolves the executor selected by the launch profile, waits for the required action server, dispatches the validated goal, tracks step state, and accepts only typed results. It subscribes to `/intents` for both direct and planner-generated work. A direct intent is normalised, checked against duplicate and argument guards, and mapped to a registered execution path. When the intent is an explicit speech operation such as `say` or `greet`, the `/nao/say` action is called. Ordinary conversational responses do not cross this path and remain under `dialogue_manager`;

report_result is an execution skill whose evidence-derived text is forwarded through the report and say skill servers.

Planner-mediated work crosses two named APIs. First, **chatbot_llm** publishes a candidate planner request on `/nao_orchestrator/planner_request`; the planner gate normalises its request kind and goal lineage, stores the request context, and forwards the admitted planner request on `/planner/request` (Contract 2). Second, **planner_llm** returns the planner output on `/intents` as a nested plan envelope (Contract 5). The orchestrator checks its metadata, ordered steps, skill-registry names (Contract 4), arguments, dependencies, and failure policies before emitting **plan_accepted**. This two-sided role explains why the node appears at both admission and execution positions in Figure 5.1: one lifecycle component enforces planner admission and deterministic execution; it is not a second planner.

Accepted plans have their steps dispatched in order through the ROS action or skill-server interface mapped by each registry entry. The orchestrator retains the active goal, plan version, current step, completed results, and pending work. It publishes **step_started**, **step_succeeded**, **step_failed**, and **plan_completed** on `/planner/execution_feedback` (Contract 7), preserving `goal_id`, `plan_id`, `plan_version`, and `step_id`. This continuous execution state makes a run reconstructable and allows the planning-state helper to reject stale callbacks or respond to a recoverable failure without moving robot control into the planner.

On failure, the orchestrator reports the typed skill result (Contract 6) and stops, continues, or waits according to the validated policy. It does not ask an LLM for an unregistered action, create a replacement plan, or fabricate success. Planner communication arrives on `/planner/dialogue_act` and is relayed on `/nao_orchestrator/planner_dialogue_act` (Contract 8), after which wording and speech remain under the dialogue layer. The registered **report_result** path is the explicit execution-speech case: the orchestrator derives its content from Contract 6 evidence and dispatches it through the report and say skill servers. These restrictions implement FR2, FR3, FR5, FR6, and FR7 by keeping route ownership, planning, execution, evidence, and speech authority at their declared seams.

Figure 5.4 shows the two ingress paths and the subsequent skill dispatch, KB-effect application, execution feedback, and dialogue-act relay.

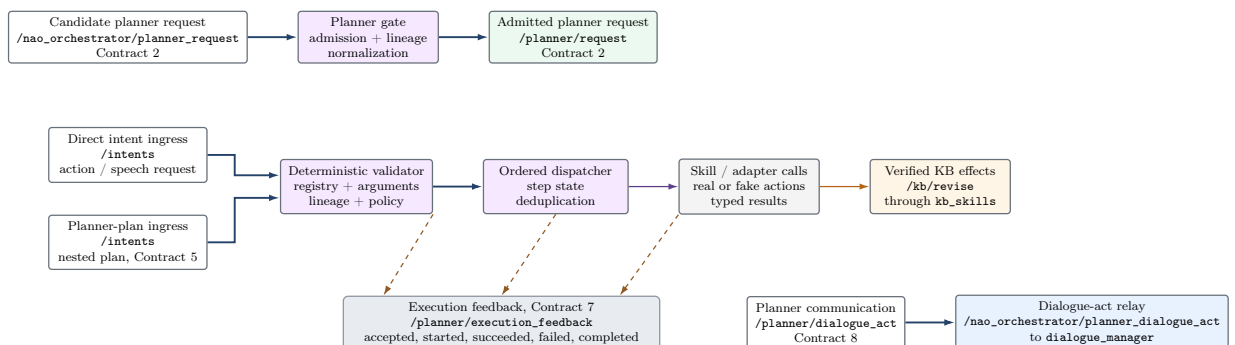


Figure 5.4: The `nao_orchestrator` admission and execution boundaries. All effects cross deterministic validation and retain goal and plan lineage.

5.3 Deterministic Guards and Bounded Fallbacks

Model output is the primary source of dialogue routing and planner decisions, while deterministic code constrains the interfaces around it. The guards are implemented as enumerated route labels, lexical marker sets, schema and type checks, skill and alias maps, required-argument checks, lineage comparisons, deduplication signatures, and allowed failure-policy constants. They handle malformed or contradictory edge outputs at the last responsible boundary. Every repair or fallback is trace-visible, and none may convert missing evidence into a successful result. The severe-fallback count in Chapter 6 measures whether a run depended on these exceptional paths.

At the chatbot boundary, `chatbot_llm` maps model labels to the three allowed routes, compares the response-stage route with the intent-stage route, and checks the utterance and structured payload for a small set of execution and KB-mutation markers. A contradictory acknowledgement may therefore be sanitised or the route may be repaired to the only contract-compatible value. If the fields remain inconsistent, the turn ends as a typed failure and no candidate request is published on `/nao_orchestrator/planner_request`. In `intent_first`, the locked route is carried into response generation and execution wording is checked against it. These constant-driven checks enforce Contracts 1 and 2; they do not infer a plan.

At the planner boundary, `planner_engine` calls the configured model and extracts the first JSON object. It then normalises the plan envelope and checks step types, registered skill names, arguments, dependencies, failure policies, requested reporting, and lineage against Contracts 4 and 5. A valid `clarify` or `fail` decision is accepted as a planner outcome. Invalid JSON or an invalid executable plan triggers one contract-repair model call containing the validation errors and a bounded copy of the previous output. If that retry is valid, its normalised result is used. If it is still invalid, only tightly scoped compatibility repairs may run. These use hard-coded lexical conditions and already grounded entity groups, remain subject to the same registry and plan validator, and are marked in planner decision metadata. They are retained as edge-case resilience rather than claimed planning capability. When no repair applies, the node emits a typed invalid-planner-output failure; it does not ask the user to clarify a schema error or dispatch a partial plan.

At the execution boundary, `nao_orchestrator` resolves aliases through the skill registry, compares each argument with the registered requirements, verifies goal and plan lineage, rejects duplicate event signatures, and applies only an enumerated failure policy before dispatch. Its `report_result` path derives the final summary from successful result payloads or trace evidence, so the planner cannot predeclare an effect that the skill did not observe. Profile-dependent selection between real and fake adapters changes the receiving action server, not planner policy. Skill results follow Contract 6, feedback on `/planner/execution_feedback` follows Contract 7, and clarification or completion requests cross the Contract 8 relay before speech.

5.4 World State and Scene Grounding

The perception path has parallel people and object branches. The person-perception branch uses `hri_face_detect_yunet` to detect faces and `hri_person_manager` to maintain person identities, tracked-person topics, and frame-qualified person state. The object branch uses `nao_scene_grounding` to convert detector output into a provider-independent scene representation. A custom YOLO detector was trained for a restricted proof-of-concept object set and integrated with the NAO camera stream through ROS. Its strongest demonstration classes are blueberry, corn, pear, tomato, and zucchini. Detector backends are normalised into one internal observation shape, so downstream nodes do not depend on the detector message type.

The grounding node filters object detections by confidence and an explicit label allowlist, maps labels to KB classes, and maintains local identifiers using tracker IDs or bounded image-plane matching. Without a tracker ID, the default matcher requires the same label, KB class, and detector source, a displacement of at most 64 pixels, and an observation age of at most two seconds. It publishes `/scene/summary` and writes accepted observations as transient KnowledgeCore statements through `/kb/revise`. Under the default profile, KB observations have a four-second lifespan and are refreshed no more often than once per second; a local object is removed from the current scene summary after 4.5 seconds without a new detection. Therefore, an object marked visible in `grounded_context` is present in this bounded current-scene window, not merely stored indefinitely in the KB. People remain owned by the tracked-person pipeline and are merged only when the compact symbolic context is assembled. This implements the RDF and evidence ownership in Contract 9, while Contract 3 exposes the bounded `grounded_context` projection used by the LLM nodes.

`kb_skills` is the transport boundary for `/kb/query` and `/kb/revise`. The chatbot first formats the bounded query result as an internal knowledge snapshot, combines it with the current scene summary and tracked-person state, and emits `grounded_context` (Contract 3). This is the only compact spatial-symbolic contract passed to the LLM stakeholders. It keeps entities, locations, and bounded relations while omitting unrestricted raw rows and detector payloads. Tracked people may already expose frame-qualified transforms through `hri_person_manager`, which supports relative person visualisation in `interaction_sim`. Object `center_x` and `center_y` remain image-plane evidence and are not interpreted as metric distance. Metric object position requires an RGB-D, stereo, simulator, or other pose source to publish a timestamped 3D observation with camera calibration, frame identity, transform, and uncertainty on the spatial-overlay seam.

Planning context and execution evidence consequently have different roles. Current symbolic state is sufficient for a non-mutating knowledge answer and useful for selecting a plan. It is not sufficient to claim that an object was found after a scan, that a path remains clear, or that a manipulation succeeded. A skill therefore returns a Contract 6 payload containing the evidence it observed and, when it materially changes the relations of a KB entity, explicit `evidence.kb_effects`. Successful fake `pick_object`, `place_object`, and `bring_object` results remove stale holding or source-location relations and add the new support, destination, or recipient relation through the same interface. The orchestrator applies those effects through

`kb_skills` and verifies the skill post-conditions through `/kb/query` when available. Failure to update or verify `kb_effects` fails the step rather than allowing later dialogue to report the intended effect as current state.

Figure 5.5 shows how the object and person perception branches feed KnowledgeCore, scene summaries, and grounded context. It also marks the future spatial-overlay boundary.

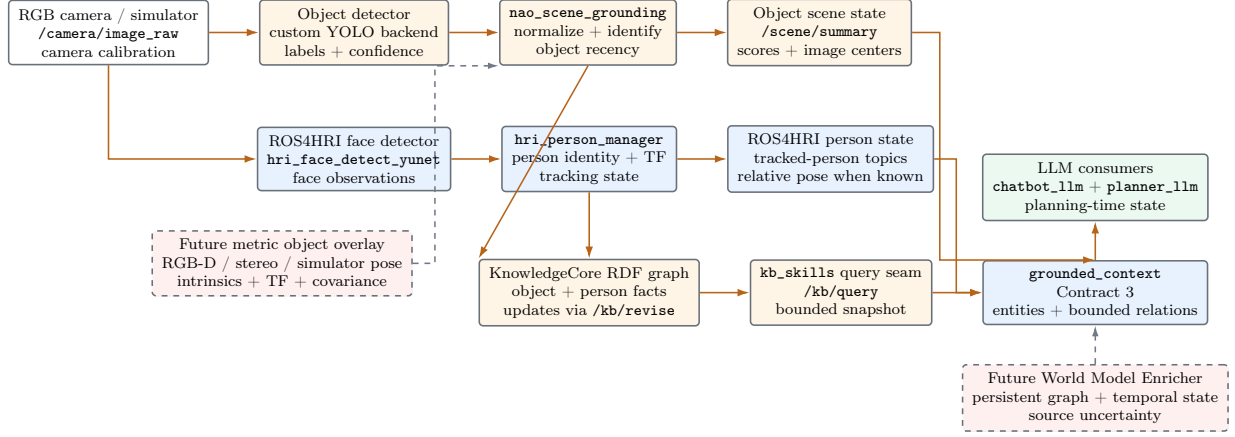


Figure 5.5: Combined perception and symbolic-grounding pipeline. Compact projections serve dialogue and planning, while raw and fresh evidence remains at its source interface.

5.5 Fake Skills and Simulation

The `fake_skills` package supplies deterministic action servers for the validation routes identified in Table 5.1. It preserves the Contract 6 result shape while replacing hardware execution with an explicit policy. Table 5.2 lists the order in which the server selects that policy. The first non-empty source wins, so a case can override one call without changing the launch-wide policy. Each result records `metadata.fake=true`, the scenario and result mode, its policy source, deterministic seed, and simulated duration. Dispatch and outcome events are also published on `/fake_skills/events`.

Table 5.2: Fake-skill outcome-policy resolution order.

Order	Policy source	Configuration surface	Recorded marker
1	Per-call scenario override	<code>scenario_override.result_mode</code> supplied by the test case	<code>mode_source=scenario_override</code>
2	Per-call action result mode	The action goal's <code>result_mode</code> field	<code>mode_source=request_args</code>
3	Per-skill override	Launch parameter <code>mode_overrides_json</code> , keyed by skill name	<code>mode_source=skill_override</code>
4	Global policy	<code>always_success</code> , <code>always_fail</code> , <code>every_other</code> , or <code>random_seeded</code>	<code>mode_source=global_mode</code>
5	Named scenario or default	Active scenario configuration; otherwise the skill's typed success mode	<code>mode_source=scenario_default</code>

This server lets the same planner and orchestrator path exercise all-success, fail-once, blocked-

delivery, missing-recipient, and ambiguous-evidence cases. Stateful modes use deterministic call counters, and `random_seeded` uses the recorded seed and failure probability. Successful manipulation results can emit the KB effects described above, so later steps and turns observe a changed scenario state rather than a fixed textual fixture. The fake package implements selected Contract 4 entries behind the executor mappings chosen by the launch profile, providing the semantic result and failure surface of the skill without claiming a physical effect.

The fake skill servers can therefore be used for stack-level validation. A fake-skill pass supports claims about routing, plan validity, deterministic dispatch, feedback handling, KB post-effects, and recovery policy. It does not demonstrate physical navigation accuracy, grasp success, detector quality, NAOqi timing, or human-robot safety. Chapter 6 reports the multi-step deterministic fake-skill score separately from the main speech-ingress stack and identifies robot- or detector-dependent checks explicitly.

5.6 Models and Runtime Configuration

The `nao_chatbot` package provides two main launch profiles for the integrated stack. Both start the dialogue, chatbot, planner, orchestration, grounding, skill, and observability components through the same launch builder, but select different perception and robot interfaces.

Table 5.3: Principal integrated launch profiles used by the implementation.

Launch profile	Default runtime source	Primary use
<code>nao_chatbot_sim.launch.py</code>	<code>interaction_sim</code> perception and tools, deterministic fake skills, simulated gaze by default	Dialogue, planner, grounding, ordered dispatch, failure injection, and preloaded-environment validation without requiring the robot.
<code>nao_chatbot_robot.launch.py</code>	Packaged NAO robot description and TF tree, robot camera, HRI visualization, RViz, detector and scene grounding	Robot-camera and frame validation, real gaze and motion adapters, detector-backed KB entities, and guarded hardware execution.

The principal launch controls are grouped by profile:

- The simulator profile exposes `start_interaction_sim`, `start_interaction_sim_perception`, `start_interaction_sim_tools`, `start_fake_skills`, `preloaded_environment_ids`, and per-skill execution modes.
- The robot/RViz profile exposes `start_ nao_robot`, `start_rviz`, `start_object_detection`, `object_detection_backend`, and `start_scene_grounding`. Simulator tools may remain active while simulator perception is disabled, preventing synthetic camera data from competing with the robot stream.

Both profiles expose `chatbot_turn_pipeline_mode`, the CLI argument to control the response, route and intent generation of the chatbot as well as detector thresholds, timeouts, and other ROS runtime parameters.

The current defaults use the `Qwen3-VL-30B-A3B-Instruct-AWQ` model for both chatbot and planner inference through an OpenAI-compatible vLLM endpoint, with model reasoning disabled for the scored path. The planner default uses temperature 0.1 and an 800-token output bound; the chatbot uses separate response and intent token limits. The same launch arguments can select an Ollama provider or different models without changing node contracts.

ROS lifecycle sequencing is part of the runtime configuration. The launch builder configures and activates `chatbot_llm`, `dialogue_manager`, `nao_orchestrator`, and the robot-facing `say`, `scan`, and `report-result` servers before declaring the dialogue path ready. `planner_llm` runs as a standard `rclpy` node with provider preflight checks and explicit failure visibility. Final validation also records the selected response mode, active real or fake adapters, detector backend, environment fixture, and launch arguments. These facts form the reproducibility bundle defined in Chapter 6.

Chapter 6

Validation Methodology

6.1 Validation Scope and Unit of Analysis

The evaluation tests whether planner-mediated interaction preserves the runtime contracts defined in Chapter 4. A successful case must route the user turn correctly, use the declared grounded evidence, produce an admissible plan when execution is required, dispatch only registered skills, retain execution lineage, and report the observed outcome through the designated dialogue and speech interfaces. Task completion alone is insufficient when it is accompanied by duplicate speech, unsupported world-state claims, invalid planner output, or missing trace evidence.

The unit of analysis is one case execution under a fixed configuration tuple:

$$\mathcal{C} = (u, p, e, m_c, m_p, h_c, h_p, f, s),$$

where u is the user utterance, p the launch profile, e the environment fixture, m_c and m_p the chatbot and planner models, h_c and h_p the corresponding prompt-pack revisions, f the real or fake execution policy, and s the deterministic seed when one is used. A run is one complete case-set execution under a shared tuple. Results from different tuples are reported as separate evidence sets rather than merged into one aggregate.

The independent variables are the turn-pipeline mode, case set, fixture, detector state, skill-adaptor profile, injected failure policy, seed, and model configuration. The dependent variables are route correctness, plan validity, required-skill coverage, order correctness, safe failure, clarification behaviour, phase completeness, speech multiplicity, state freshness, latency, and the run-level score defined in Section 6.11.

6.2 Validation Levels and Supported Claims

The runtime harness evaluates five levels of evidence. Passing one level does not imply the next: a turn may traverse every expected ROS interface while preserving the wrong target set

or an inconsistent symbolic post-state. Table 6.1 defines the evidence required at each level and the claim it can support.

Table 6.1: Validation levels and supported claims.

Depth	Required evidence	Permitted claim
Runtime readiness	Source and image fingerprint, unique nodes, active lifecycle state, speech QoS, model preflight, KB services, and required action servers.	The declared configuration was available for case injection.
ROS trajectory	User turn, route, planner admission when eligible, plan or dialogue outcome, execution feedback, terminal event, and speech.	The case traversed the expected software path.
Semantic contract	Exact requested and selected members, entity roles, plan targets, recipient, ordering, and report policy agree.	The executed software task was a valid refinement of the user request.
State and closure	Per-member effects pass KB post-queries and the final structured outcome covers every evidenced success and failure.	The symbolic post-state and user-facing closure agree with execution evidence.
Embodied outcome	Robot, scene, or sensor evidence proves the physical effect under the detector or robot profile.	The corresponding physical behaviour was observed within the evaluated conditions.

Deterministic fake skills can support readiness, trajectory, semantic-contract, and symbolic post-state claims. They do not establish physical navigation or manipulation accuracy. The principal semantic-stack score therefore stops at state and closure. Detector-enabled and physical-robot evidence are reported separately.

6.3 Execution Profiles

Table 6.2 distinguishes the scored full-stack profile from diagnostic and ablation profiles. This prevents planner-isolated success or detector variability from being reported as integrated-stack performance.

Table 6.2: Evaluation profiles and their validation scope.

Profile	Entry point	Components under test	Score use
E2E-main	Tracked voice and <code>hri_msgs/LiveSpeech</code>	Dialogue lifecycle, chatbot routing, grounding, planner admission, execution feedback, and speech ownership	Primary full-runtime score.
Planner-isolated	<code>/nao_orchestrator/planner_request</code>	Planner gate, planner output, orchestrator validation, selected skills, and feedback	Diagnostic score only.
Fake-multistep	E2E-main with named fixtures and deterministic fake policies	Ordered execution, failure injection, replanning, clarification, KB effects, and truthful reporting	Separate fake/replan score.
Detector-enabled	E2E-main with the detector and grounding bridge enabled	Detector normalisation, scene summary, KB projection, and detector-derived dialogue	Perception score reported separately.
Intent ablation	E2E-main with <code>intent_first</code>	Route lock, response consistency, planner admission, and speech behaviour under changed stage ordering	Paired ablation against <code>response_first</code> .
Architecture sweep	Chatbot service, KB services, and direct fake actions	Contract-specific add/revise/remove, no-mutation, skill guard, and adapter checks	Compliance evidence, not a substitute for E2E-main.
Direct atomic control	Direct <code>/intents</code> ingress for one registered atomic operation	Orchestrator normalization, argument guards, dispatch, and typed result handling without planner mediation	Safety control only; not a causal planner baseline.
Source verification	Focused unit, contract, and launch tests	Schema normalization, semantic validators, registry projections, and deterministic guards	Implementation evidence only; no live-runtime score.

The primary E2E profile uses `response_first`, group-scoped synthetic voices, and the tracked-voice plus `LiveSpeech` ingress. Unrelated cases receive separate conversation groups, while intentional follow-ups share a declared group. The detector is disabled in the principal semantic-stack score and enabled only for the detector profile. Stable KB-authored objects are therefore not confused with object-recognition performance.

6.4 Validation Goals and Acceptance Rules

Each goal in Table 6.3 has an explicit pass rule. Evidence inferred only from a timeout, a console summary, or a topic that was not observed does not satisfy the corresponding rule.

Table 6.3: Validation goals, acceptance rules, and evidence sources.

Goal	Pass rule	Evidence source
Route correctness	The observed route equals the declared route. Dialogue and knowledge turns emit no planner request or execution feedback.	<code>/chatbot_llm/turn_trace</code> , <code>planner-request</code> and <code>intent</code> topics.
Semantic-decision uniqueness	One user turn produces one validated route before acknowledgment. A dialogue response followed by unintended execution is a failure even when both paths are individually observable.	Turn identifier, route decision, planner request, dialogue response, and speech events.
Plan validity	Every candidate plan admitted for execution passes Contract 5 schema, Contract 4 skill-name, argument, ordering, failure-policy, and lineage checks.	Planner decision record, <code>/intents</code> , orchestrator validation events.
Skill and order coverage	All required subgoals appear, forbidden skills are absent, and declared partial-order constraints hold.	Expected-plan manifest and observed plan steps.
Grounding use	Required fixture entities appear in Contract 3 using canonical identifiers and the declared people/object/location roles.	KB pre-query, <code>/scene/summary</code> , planner request, grounded-context trace.
Target-set and role integrity	The selected, planned, executed, and reported members equal the declared object set. People remain recipients or human targets; locations and supports remain anchors.	Case oracle, <code>target_selection</code> , admitted plan, execution events, and structured report outcome.
Execution validation	Only admitted steps are dispatched, and every dispatch has a Contract 6 result and Contract 7 feedback with matching lineage.	Action events, <code>/planner/execution_feedback</code> , trace JSONL.
Safe failure	An injected failure ends in a valid replan, clarification, help request, or truthful terminal failure. False completion is a fail.	Fake result, plan versions, dialogue act, spoken result.

Table 6.3 continued

Goal	Pass rule	Evidence source
Exactly-once speech	Each expected semantic stage produces one speech event. Acknowledgement and terminal reporting count as separate stages only when both are declared.	Planner dialogue relay, dialogue output, /debug/nao_say/speech.
Report coverage	Every selected member with execution evidence appears in the structured outcome with the correct status; the spoken closure must not claim completion when evidence is incomplete.	Contract 6 results, report outcome, report-result event, and final speech.
Trace completeness	All mandatory phases contain timestamps, identifiers, and payload evidence sufficient to reconstruct the case.	Per-case <code>phase_observations</code> , JSONL trace, report bundle.
State freshness	The pre-case fixture is verified, undeclared earlier entities are absent, and successful material effects are visible in a post-case KB query.	Fixture manifest, stale-world guard, /kb/query, skill KB effects.
Interaction-interface compliance	Public interaction follows the tracked-voice and LiveSpeech contracts; dialogue ownership, lifecycle readiness, and planner-dialogue relay remain intact.	ROS graph, lifecycle state, QoS/subscription preflight, relay topics.

6.5 Full Runtime Questionnaire

The principal questionnaire contains twenty-one cases. The original twenty cases retain five dialogue, five knowledge, five simple execution, and five composite execution cases. A route-safety holdout adds one explicit non-action turn. Table 6.4 records the exact utterances used by the runtime harness. Dialogue and KB-query cases forbid planner activity. Execution cases admit planner work but still require grounded arguments, registered skills, truthful evidence, and the speech policy declared for the case.

Table 6.4: Frozen twenty-one-case full-runtime questionnaire.

Case ID	Exact utterance	Route	Required or forbidden evidence
dialogue_greeting	"Hey, how are you?"	dialogue	One response; no planner activity.

Table 6.4 continued

Case ID	Exact utterance	Route	Required or forbidden evidence
dialogue_favorite_movie	“What is your favourite movie?”	dialogue	Natural answer; no planner activity.
dialogue_movie_followup	“My favourite movie is Back to the Future!”	dialogue	Contextual acknowledgement; no planner activity.
dialogue_weekend_plans	“Any ideas for plans this weekend?”	dialogue	Advice is not promoted to execution.
dialogue_speed_of_light	“What is the speed of light?”	dialogue	Informational answer; no planner activity.
kb_visible_now_baseline	“What can you see now?”	knowledge	Answer from declared state; no planner activity.
kb_probe_scene_update	“What else can you see now?”	knowledge	Newly injected cup/table facts are used.
kb_object_on_table	“What object is on the table?”	knowledge	Correct support relation and canonical subject.
kb_cup_name	“Can you tell me the name of the cup?”	knowledge	Name is recovered from the KB projection.
kb_multi_object_count	“How many objects can you see?”	knowledge	Correct count without treating people/supports as objects.
skill_head_up	“Move your head up.”	execution	Admissible motion and evidence-based report.
skill_pick_phone_generic	“Pick up the phone.”	execution	Grounded <code>pick_object</code> ; noun phrase is resolved before dispatch.
skill_pick_object_on_table	“Pick up the object on the table.”	execution	Support relation identifies one canonical target.
skill_look_at_person	“Look at the person named ALEX.”	execution	Person remains a human target; <code>look_at</code> is dispatched.
kb_mutation_add_red_cup	“Add a red cup to your KB.”	execution	Explicit <code>kb_add</code> through the KB boundary and post-query.
composite_head_all_directions	“Can you move your head in all directions?”	execution	Multi-step motion; no one-step collapse.
composite_bring_every_object_to_person	“Can you bring every object in view to the person named ALEX?”	execution	Canonical objects and human recipient; complete report.
maximal_kitchen_cup_to_operator	“Can you go to the kitchen and bring me the cup?”	execution	Grounded location/object chain; no fabricated detector evidence.
composite_go_to_person_wave_report	“Go to the person named ALEX and wave at them. Let me know when you’ve done that.”	execution	Ordered navigation, social motion, and terminal report.
composite_walk_every_object_reports_main	“Walk to every object and let me know when you get to each one.”	execution	Alternating navigation/report stages for every grounded object.
dialogue_non_action_recipient_memory	“Remember that ALEX is the recipient, but do not act yet.”	dialogue	One acknowledgement; no planner request, KB mutation, or execution feedback.

6.6 Multi-Step Deterministic Fake-Skill Matrix

The fake-skill matrix reuses the full tracked-voice and LiveSpeech ingress and replaces hardware-dependent outcomes with named, typed policies. The canonical manifest contains the nine cases in Table 6.5. Each policy sweep retains the same prompts and fixtures, allowing

differences to be attributed to injected outcomes rather than changed language.

Table 6.5: Canonical multi-step deterministic fake-skill manifest.

Case ID	Fixture	Exact utterance	Acceptance focus
fake_deep_baseline_inventory	baseline_table	“What objects are on the table?”	Fixture and grounding preflight.
fake_deep_ordered_walk_report	lab_table	“Walk to every object on the table and let me know when you get to each one.”	Ordered reports and no repeated steps.
fake_deep_grouped_work_table_delivery	lab_sections	“Bring every object from the work table to ALEX and report what happened.”	Group expansion, recipient, and KB effects.
fake_deep_missing_object_recovery	baseline_table	“Find the codex missing cup. If you cannot find it, scan the scene and report what you can confirm.”	Replan or truthful failure.
fake_deep_missing_recipient_clarification	lab_sections	“Bring every object from the work table to the person named BLAKE and report what happened.”	No invented recipient or false completion.
fake_deep_iiia_kitchen_delivery	iiia_floor	“Bring every object from the kitchen to ALEX and report what happened.”	Maximal location-scoped delivery.
fake_deep_iiia_floor_location_followup	iiia_floor	“Where are the objects from the kitchen now?”	Post-effect state, not stale origin facts.
fake_deep_gold_apple_multiturn	gold_apple_handoff	“Can you bring that gold apple to the person named ALEX?”	Pronoun, colour, recipient, and delivery.
fake_deep_gold_apple_followup	gold_apple_handoff	“Where is the gold apple now?”	Follow-up from verified KB state.

The deterministic policy ladder is `all_success`, `fail_once_navigation`, `fail_once_pick`, `delivery_blocked`, and `recipient_missing`. A seeded pseudo-random profile is used only after these policies have been completed, and its seed is part of the run tuple. An isolated rerun may diagnose a failed case, but it does not overwrite the full-ladder result; both outcomes are reported.

6.7 Robustness Set and Repetition

Five language variants are frozen after source hardening and excluded from subsequent implementation selection. They test a paraphrased grouped delivery, explicit exclusion of person and support targets, complete whole-chain reporting, a missing recipient, and a dialogue-only future instruction followed by an explicit execution turn. The compact set is repeated three times with independent dialogue groups. The exhaustive main, environment, and deterministic-policy profiles are run once; every degraded or failed case is repeated once in isolation. The full-sweep status remains primary, while the isolated result diagnoses case interaction or nondeterminism.

Table 6.6: Frozen five-case robustness set.

Case ID	Exact utterance	Acceptance focus
robust_paraphrased_group_delivery	“Take everything resting on the work table over to ALEX. Once every item has reached them, summarize the whole delivery.”	Exact members, recipient, and final coverage.
robust_object_role_exclusion	“Visit each movable item on the table in turn. Do not navigate to the table or to any person, and report each arrival.”	Object-only selection and per-target reports.
robust_missing_recipient	“Deliver every work-table object to MORGAN and explain what information is missing if MORGAN is not present.”	Truthful clarification without execution.
robust_future_recipient_memory	“ALEX will receive the work-table items later. For now, only acknowledge this and do not execute anything.”	Dialogue-only decision with no KB mutation.
robust_multiturn_execute_followup	“Now deliver all of those objects to them and tell me the overall result.”	Grounded multi-turn carry-over and complete report.

6.8 Interface and Architecture Checks

The following preflight checks are mandatory before any speech-mode case is scored:

1. the container fingerprint, source revision, image tag, and model configuration are recorded;
2. `dialogue_manager` is in the active lifecycle state and subscribes to the remapped speech topic;
3. the tracked-voice topic uses reliable transient-local delivery and the per-voice `LiveSpeech` topic uses reliable volatile delivery;
4. the planner ingress follows `chatbot_llm -> nao_orchestrator -> planner_llm`;
5. planner communication is published on `/planner/dialogue_act`; `nao_orchestrator` relays it and `dialogue_manager` realizes the utterance;
6. long-running skills expose action servers, streamed state remains on topics, and short KB operations remain behind `kb_skills`;
7. no duplicate nodes, stale container imports, or unconfigured required lifecycle skill servers are present.

A preflight failure marks the run as *not scored*; it is not attributed to chatbot, planner, or model behaviour. NAO-specific transport remains behind adapters, people and objects remain semantically distinct, and planner or orchestrator code must not bypass the public tracked-voice, dialogue, and speech path.

6.9 Reset, Repetition, and Isolation

Before each case group, the harness clears terminal goal state, resets fake-skill counters and policy, injects the declared fixture through `/kb/revise`, verifies the fixture through `/kb/query`, checks the stale-world guard, and begins a new trace interval. Run-unique fixture identities are preferred when equivalent semantic names may recur. Cleanup checks outgoing

and incoming asserted relations, waits for KnowledgeCore materialisation to settle, and refuses to score the next group when absence cannot be verified. Successful state-changing fake steps are checked against their declared per-member postconditions. The dialogue scope is new unless the case explicitly tests a follow-up. Fixture lifetimes must exceed the maximum group duration.

One full questionnaire or policy sweep is the primary run unit. Any degraded or failed case is repeated once in isolation using the same tuple to distinguish case interaction from intrinsic failure. The isolated result is reported beside the full-sweep result. Latency statistics are computed only for case families with at least three valid observations. Infrastructure failures before turn injection may be retried once and are logged as exclusions; failures after injection remain in the denominator.

6.10 Metrics and Denominators

Let N be the number of declared cases, including cases that fail or do not reach a terminal event. The principal rates are:

$$\text{RouteAccuracy} = \frac{N_{\text{correct route}}}{N}, \quad (6.1)$$

$$\text{ValidPlanRate} = \frac{N_{\text{plans accepted}}}{N_{\text{planner eligible}}}, \quad (6.2)$$

$$\text{SkillCoverage} = \frac{\sum_c |S_c^{\text{required}} \cap S_c^{\text{observed}}|}{\sum_c |S_c^{\text{required}}|}, \quad (6.3)$$

$$\text{OrderAccuracy} = \frac{N_{\text{cases satisfying required order}}}{N_{\text{ordered cases}}}, \quad (6.4)$$

$$\text{SafeFailureRate} = \frac{N_{\text{safe failure outcomes}}}{N_{\text{injected failure cases}}}, \quad (6.5)$$

$$\text{PhaseCompletion} = \frac{\sum_c N_{c,\text{observed mandatory phases}}}{\sum_c N_{c,\text{mandatory phases}}}, \quad (6.6)$$

$$\text{ExactlyOnceSpeech} = \frac{N_{\text{semantic stages spoken once}}}{N_{\text{expected speech stages}}}. \quad (6.7)$$

The semantic rates are:

$$\text{TargetSetExactness} = \frac{N_{\text{cases with exact selected,planned,executed,reported sets}}}{N_{\text{target bearing cases}}}, \quad (6.8)$$

$$\text{RoleIntegrity} = \frac{N_{\text{role sensitive cases without forbidden kinds}}}{N_{\text{role sensitive cases}}}, \quad (6.9)$$

$$\text{ReportCoverage} = \frac{N_{\text{selected members represented in the structured outcome}}}{N_{\text{selected members requiring reporting}}}, \quad (6.10)$$

$$\text{PostconditionRate} = \frac{N_{\text{verified required effects}}}{N_{\text{declared required effects}}}, \quad (6.11)$$

$$\text{DecisionUniqueness} = \frac{N_{\text{turns with one validated semantic decision}}}{N_{\text{injected turns}}}. \quad (6.12)$$

Clarification precision and recall are reported separately. A clarification is incorrect when the fixture already supplies the required object, location, or recipient. State freshness is the proportion of cases whose pre-case guard is clean and whose declared post-effects pass the post-query. Latency is reported with median, interquartile range, 95th percentile, maximum, and timeout count; the mean is supplementary.

Case status uses four values. *Pass* means that all mandatory observations and the declared terminal behaviour are present. *Degraded* means that the behaviour is truthful but a non-critical requirement or observation is incomplete. *Fail* includes a wrong route, invalid admitted plan, missing mandatory phase, unsafe recovery, false success, duplicate semantic speech, or incorrect state transition. *Not scored* is reserved for a failed preflight or an artifact that never reached case injection. The weighted case rate is

$$\text{WeightedCaseRate} = \frac{N_{\text{pass}} + 0.5N_{\text{degraded}}}{N};$$

cases marked not scored remain in N , so incomplete coverage cannot increase the rate.

6.11 Runtime Review Score

The thesis reports the runtime-review 1–10 score as a secondary operational index. Reproducible case, semantic, state, speech, and latency metrics remain the primary results. Table 6.7 defines eight weighted dimensions. For dimension j , the compliant fraction r_j is calculated from its declared cases or mandatory observations and lies in $[0, 1]$. Every published index includes the eight component values and their denominators.

Table 6.7: Components of the run-level reviewer score.

Dimension	Weight	Principal evidence
Dialogue and route safety	1.0	Route accuracy, non-execution of dialogue/KB turns, and response quality.
Grounding and KB state	1.5	Fixture use, canonical identities, state freshness, and verified KB effects.
Plan admission	1.5	Valid-plan rate, registered skills, arguments, ordering, and lineage.
Skill execution	1.5	Required-skill coverage, action results, terminal behaviour, and truthful reporting.
Failure and replan handling	1.5	Safe-failure rate, clarification correctness, plan-version progression, and recovery.
Speech ownership	1.0	Exactly-once semantic stages and absence of raw planner/backend text.
Trace completeness	1.0	Mandatory phase observations, identifiers, timestamps, and attributable outcomes.
Runtime health	1.0	Lifecycle/QoS preflight, node uniqueness, bounded latency, and absence of sustained resource pressure.

The uncapped score and reported score are

$$S_{\text{raw}} = \max \left(1, \sum_j w_j r_j \right), \quad S_r = \min (S_{\text{raw}}, C_r),$$

where the weights w_j sum to 10 and C_r is the critical-failure cap. Component values are retained at full precision and S_r is rounded to one decimal place. A simple-dialogue failure, duplicate semantic speech, raw planner leakage, or false execution success sets $C_r = 6$. Missing mandatory evidence or failure of stable simulator/KB grounding sets $C_r = 7$; otherwise $C_r = 10$. Detector-only variability is reported under the detector profile and does not cap the semantic-stack score. Route-repair and fallback counters are pressure telemetry: they lower a component only when they produce an incorrect route, invalid admitted plan, missing phase, unsafe recovery, or user-visible failure. A score of 9 or above additionally requires complete speech ingress, natural-chat KB mutation, deterministic failure/replan evidence, exactly-once speech, and a complete fingerprinted evidence bundle.

6.12 Research-Question Evidence Map

Table 6.8 links each research question to its primary profile and metrics, and states how far the resulting evidence can support the claim. A diagnostic probe or software-only fake-skill result cannot therefore be reported as broader embodied evidence.

Table 6.8: Research questions, primary profiles, and acceptance evidence.

RQ	Primary profile	Principal metrics	Claim boundary
RQ1	E2E-main and fake-multistep	Valid-plan rate, skill coverage, order accuracy, and target-set exactness	Coherent plans for the frozen task families, not unrestricted natural-language planning.
RQ2	E2E-main and environment	Grounding use, role integrity, state freshness, and postcondition rate	Consistency with injected or observed state within the declared fixture lifetime.
RQ3	Planner admission and architecture sweep	Invalid-plan rejection, registered-skill coverage, and unsupported-dispatch count	Deterministic rejection and bounded repair, not proof that the model never proposes invalid output.
RQ4	Deterministic failure policies	Safe-failure rate, plan-version lineage, clarification precision/recall, and truthful closure	Recovery under the injected policies and available skills.
RQ5	E2E-main and direct atomic control	Route accuracy, decision uniqueness, trace completeness, exactly-once speech, and ownership checks	Inspectability and separation within the evaluated NAO stack.

6.13 Evidence Bundle

Every scored run produces a reproducible, hash-addressed bundle:

Listing 6.1: Validation evidence bundle.

```

1 docs/evaluation/runs/<run_id>/
2   manifest.json
3   per_case_metrics.csv
4   aggregate_metrics.json
5   raw/
6     <questionnaire>.json
7     <snapshot>.json.gz
8     <selected runtime logs>

```

The manifest records hashes for the original artifacts and bundled copies, source and nested-repository revisions, dirty-diff status, image identity, model and provider settings, prompt-pack and registry hashes, turn-pipeline mode, digest setting, detector state, fake policy, seed, and lifecycle/QoS preflight. Each case records its emitted utterance, fixture, route, target selection, plan, skill events, Contract 6 results, Contract 7 feedback, KB pre/post queries, speech stages, fallback markers, timestamps, and terminal assessment. Historical or focused artifacts that lack this fingerprint are labelled qualified or diagnostic and cannot define the final score.

6.14 Threats to Validity

Deterministic fake skills evaluate software supervision, recovery, and declared symbolic effects, not physical navigation or manipulation accuracy. A fixed questionnaire samples only part of the language space, and repeated development against known prompts can tune behaviour to the evaluation set. The frozen robustness set reduces but does not remove this risk. Model output may vary despite fixed prompts and low temperature. Results from different model, source, or prompt tuples are therefore not pooled. Fixture leakage, expiring facts, reused conversation history, and mutable container tags can confound a run; the reset and fingerprint gates are intended to expose these effects. A trajectory pass can also overestimate task correctness when semantic targets or postconditions are not inspected, which is why the semantic and state depths remain separate. Finally, a single NAO platform limits claims about transfer to other robots, while detector-enabled results depend on camera conditions and the restricted proof-of-concept object classes described in Chapter 5.

Chapter 7

Results

7.1 Selected Evidence Sets

Each evidence set fixes its launch mode, model or source configuration, case set, artifact, and validation status. Scores from different tuples are not averaged because a model can produce fluent dialogue while reducing planner schema validity or recovery reliability. Case metrics are the primary results; the runtime-review score is retained as a secondary run summary.

The recovered evidence contains four historical sets and two later diagnostic sets. The 28 June response-first run supplies the broad full-stack baseline. The 8 July response-first run supplies the deterministic fake/replan matrix under JSON-only grounded context. The 10 July audit applies stricter KB-effect and report-result checks. The 11 July run is an alternate-model ablation. The 13 July evidence is reported under both its original trajectory rubric and a later semantic audit. The 14 July artifacts are focused post-hardening probes and do not form a complete run tuple. None of these sets satisfies the clean-source fingerprint gate. Consequently, no cross-run aggregate is inferred from them.

Table 7.1: Run-level and diagnostic evidence sets.

Run	Profile	Scope	Evidence summary	Standing
F28	<code>response_first</code>	Full runtime	Speech ingress, KB add/revise/remove, simple and composite skills, grouped delivery, direct fake guards, and KB post-effects.	Historical, 8.7
F08	<code>response_first</code> , JSON-only grounding	Fake/replan	Four eight-case deterministic policy sweeps through tracked-voice and <code>LiveSpeech</code> ingress, with isolated confirmation probes.	Historical, 8.6
F10	Strict seam audit	KB/report hold-out	Natural KB mutation, grouped delivery effects, follow-up state, and replan/report checks.	Qualified, 6.7
F11	Alternate-model ablation	Main plus fake/replan	Fluent main questionnaire output, followed by planner-schema and deterministic-recovery regressions.	Ablation, 5.8
F13	Semantic re-audit	Full and focused suites	The main harness reported 20/20 trajectories; adversarial review found wrong target roles, incomplete closure, stale effects, and dialogue-to-execution leakage.	Diagnostic, cap 6.0
F14	JSON-only targeted hardening	Ordered and grouped cases	Exact target selection and whole-chain reporting passed focused cases; gold-apple handoff retained an incomplete-selection failure.	Diagnostic, no score

F28 and F08 are recorded in the full-suite and deterministic-fake-suite validation ledgers, including the source artifact identifiers and case-level assessments. F10 and F11 are preserved as the repository reports `runtime_review_2026-07-10_sv_seams.md` and `runtime_review_2026-07-11_qwen_personal_stack.md`. F13 and F14 are preserved under `docs/evaluation/runs` with source and bundled hashes. Their manifests mark both sets as diagnostic because their dirty-source or multi-image provenance prevents final-run status. The scores in Table 7.1 are the operational summaries recorded with the corresponding evidence; case counts and seam-level observations remain primary.

The packaged F13 diagnostic aggregate contains 54 case assessments: 39 pass, 5 degraded, 4 fail, and 6 not scored. The packaged F14 aggregate contains seven focused assessments: six pass and one fail. These totals preserve the recorded harness classifications. They do not promote a trajectory-level pass to semantic or embodied proof.

7.2 Trajectory and Semantic Contract Results

The 13 July main questionnaire reported 20/20 trajectory passes. Turns entered through the public speech path, routes and planner requests appeared, execution feedback was observed, and speech closed the cases. A later manual audit evaluated the same runtime family against exact entity roles, selected members, KB postconditions, and whole-chain reports. Table 7.2 shows why the two assessments are not interchangeable.

Table 7.2: Trajectory and semantic findings from evidence set F13.

Evidence depth	Result	Observation
Runtime readiness	Pass	Both model preflights passed, required lifecycle nodes were active, and focused tests passed inside the runtime image.
Main ROS trajectory	20/20 pass	The original harness observed route, execution, terminal, and speech breadcrumbs for every declared case.
Target-set semantics	Fail	An ordered-object request navigated to a kitchen, a table, and people, then reported completion.
Grouped report coverage	Fail	A multi-object delivery executed several skill steps but the final report described only the last object.
KB postconditions	Fail	Follow-up queries retained stale source-location relations after apparent delivery success.
Semantic-decision uniqueness	Fail	One explicit non-action turn produced a dialogue acknowledgement followed by unintended KB execution.

The earlier 8.2 operational assessment measured broad runtime availability under the trajectory rubric. The semantic audit imposed a 6.0 safety cap because the system could report completion for the wrong task. This is a correction in acceptance strength, not evidence that every ROS seam regressed. The result motivated exact target-selection checks, selected-set validation at final admission, report-coverage checks, stronger fixture isolation, and lineage-aware case assessment.

7.3 Focused Post-Hardening Results

Evidence set F14 contains seven targeted assessments across three successive rebuilt images, of which six passed and one failed. It is not assigned a run-level score because the artifacts do not share one complete configuration tuple. The ordered-object probe carried the exact apple, book, and phone identifiers with sequential ordering and per-target reporting. It produced three navigation results and corresponding user-facing reports without visiting the table, a location, or a person. The work-table delivery carried the cup, manual, and phone identifiers with ALEX as recipient and reported all three delivered object classes. A later kitchen

probe selected the single kitchen cup, executed delivery and report steps, and answered the post-effect follow-up. The explicit non-action recipient-memory case remained dialogue-only. The gold-apple handoff remained a negative result. The planner request exposed the apple and recipient as scene targets but carried an empty authoritative selection. The planner therefore asked for clarification rather than executing an unsafe placement. The failure is truthful, but the complete fixture should have allowed a valid handoff selection. This case remains a grounding-to-admission limitation rather than a successful recovery result.

7.4 Full Integrated Runtime Results

F28 exercised the response-first speech path from tracked voice and LiveSpeech through dialogue, planning, deterministic execution, feedback, and final speech. Table 7.3 reports the main runtime boundaries checked in that run.

Table 7.3: Full-runtime results for evidence set F28.

Validation seam	Result	Observed evidence
Dialogue routing	Pass	Simple dialogue and informational turns remained outside planner execution and produced one user-facing response.
Stable KB query	Pass	Injected cup name, colour, and support relation were available through KB query, grounded context, and dialogue.
Explicit KB mutation	Pass	Structured add, revise, query, and remove traversed the planner/orchestrator KB boundary; the post-remove query was empty.
Simple skill	Pass	Head motion and wave cases reached registered execution paths and evidence-derived reporting.
Composite skill	Pass	Head-motion plus wave and ordered multi-object navigation/report produced ordered plans and terminal reports.
Grouped task trajectory	Pass at trajectory depth	Kitchen objects and the named recipient appeared in planner work without fabricated detector evidence; exact selected-set and post-state semantics were not established by this rubric.
Fake skill guards	Pass	Canonical targets succeeded and absent or non-canonical targets returned typed failures.
Exactly-once speech	Pass	The retained speech traces contained no duplicated acknowledgement or terminal semantic event.

The run score of 8.7 reflects broad success across dialogue, grounding, execution, and KB transport while keeping detector object recognition outside the semantic-stack score. The score does not combine detector variability with KB-authored object grounding.

7.5 Deterministic Fake-Skill and Replan Results

Evidence set F08 used `response_first`, disabled the compact grounded-context digest, and exercised the same tracked-voice and `LiveSpeech` ingress under four deterministic fake policies. Table 7.4 reports the full-ladder result and the isolated confirmation result separately.

Table 7.4: Deterministic fake-skill policy results for evidence set F08.

Policy	Full ladder	Isolated probe	Interpretation
<code>all_success</code>	8/8 pass	Not required	Inventory, ordered reports, grouped delivery, maximal delivery, and multi-turn follow-up completed.
<code>fail_once_navigation</code>	8/8 pass	Not required	Failed navigation produced a newer plan version or truthful recovery without duplicate active goals.
<code>fail_once_pick</code>	7/8 pass	Failed ordered-report case passed in isolation	Recovery was correct outside the full-ladder context; the full-ladder result remains 7/8.
<code>delivery_blocked</code>	6/8 pass	Gold-apple case passed; ordered report remained failed	Delivery failure was generally truthful, but case interaction affected ordered-target extraction.

The four full sweeps produced 29 passes from 32 cases. Severe fallback markers remained zero: no duplicate active goal, invalid planner JSON, invalid executable plan, backend-unreachable speech, execution-report fallback, or rules-response fallback was recorded in the selected run. The run-level reviewer score was 8.6. Isolated passes are retained as diagnostic evidence and do not replace the corresponding full-ladder failures.

The F08 ledger records one artifact for each policy sweep and separate artifacts for the isolated ordered-report and blocked-delivery probes. Keeping the full-sweep and isolated records distinct prevents a diagnostic pass from replacing the primary 29/32 result.

7.6 Strict KB-Effect and Reporting Audit

F10 applied stricter post-condition checks than the broad success-path run. The focused ordered-walk, grouped work-table delivery, missing-object recovery, and missing-recipient clarification cases passed under `all_success`; `fail_once_navigation` also showed failed navigation, replanning, and completed recovery. The run score was reduced to 6.7 because grouped kitchen delivery reported only one delivered object, a follow-up answer retained the cup in the kitchen, and direct KB queries exposed stale source-location facts and recipient-side object-like spatial predicates.

This result distinguishes execution completion from state correctness. A successful action result is insufficient when the verified post-state still contains mutually inconsistent location relations. The finding supports the use of Contract 6 KB effects, post-query verification, and a separate state-freshness metric.

7.7 Alternate-Model Ablation

F11 evaluated Qwen3.6:27B_Q3_K_M with response-first routing and JSON-only grounded context. The twenty-case main questionnaire was marked 20/20 pass and produced fluent dialogue. The deterministic fake/replan artifacts contained 38 scored case records: 22 pass, 7 degraded, and 9 fail. The post-run snapshot recorded nine deduplicated planner-invalid-JSON events, nine invalid executable-plan events, six planner-gate rejections, and 53 route-repair events.

Table 7.5: Alternate-model ablation results for evidence set F11.

Measure	Result	Interpretation
Main questionnaire	20/20 trajectory pass	Response fluency and broad route behaviour were strong under the original trajectory rubric.
Fake/replan records	22 pass, 7 degraded, 9 fail	Structured planning and recovery reliability regressed under deterministic policies.
Planner invalid JSON	9 events	Fluent acknowledgement did not imply a valid executable plan.
Invalid executable plan	9 events	Deterministic validation prevented malformed work from reaching skill dispatch.
Planner-gate rejection	6 events	Admission guards remained observable under model pressure.
Route repair	53 events	The turn engine frequently reconciled model route inconsistencies.
Reviewer score	5.8/10	Critical structured-output and recovery failures outweighed the fluent main-suite surface.

The ablation demonstrates why the 1–10 score and the fake/replan matrix are reported alongside questionnaire pass counts. A model that answers well at the dialogue surface may still impose substantial planner repair and validation pressure.

7.8 Interaction and Runtime-Contract Preservation

Table 7.6 summarises the interaction and architecture checks applied across the response-first evidence.

Table 7.6: Interaction-interface and runtime-contract results.

Contract check	Result	Evidence
Tracked voice and speech ingress	Pass	Scored speech cases entered through tracked voice and per-voice LiveSpeech with the dialogue manager active.
Dialogue and speech ownership	Pass	Planner dialogue crossed the orchestrator relay; no selected run contained duplicate semantic speech or raw planner text.
Planner admission ownership	Pass	Execution requests crossed the orchestrator planner gate before planner consumption.
Deterministic execution boundary	Pass	Invalid plans were observable as validation or gate events rather than direct robot dispatch.
Skill actions and feedback lineage	Pass	Fake/replan traces retained goal, plan version, step results, terminal evidence, and speech evidence.
KB transport ownership	Pass	Explicit mutations traversed <code>kb_skills</code> ; ordinary questions did not mutate the KB.
Post-skill KB consistency	Degraded in F10	Stale and recipient-side spatial predicates remained after one grouped-delivery configuration.
Detector profile separation	Pass	Detector object recognition was excluded from the semantic-stack score and retained as a separate profile.

7.9 Research-Question Synthesis

Table 7.7 gives a qualified answer to each research question. Each row includes both the positive results and the failure cases that limit generalisation.

Table 7.7: Qualified answers to the research questions from the recovered evidence.

RQ	Standing	Evidence-based answer
RQ1	Supported with limits	The planner generated ordered multi-step plans for motion, navigation, reporting, and grouped delivery. F14 showed exact selected-member preservation for ordered and grouped cases, while the gold-apple case showed that this behaviour is not complete across all grounded references.
RQ2	Mixed	Stable KB insertion, name, colour, support, and count queries passed in the broad runs. F10 and F13 exposed stale or contradictory post-effects, so compact grounding supported next-turn reference but did not by itself guarantee state consistency.
RQ3	Supported within enforced contracts	Registry and deterministic validation prevented schema-invalid, registry-invalid, or lineage-invalid plans from reaching dispatch. F11 showed nine invalid JSON and nine invalid executable-plan events while the gate remained observable. The F13 counterexamples show that admission does not detect every target-selection or post-state semantic error.
RQ4	Partially supported	F08 established navigation replan lineage and truthful failure handling under deterministic policies. Pick failure, blocked delivery, and missing-recipient cases were less stable in full sweeps, and several F13 cases lacked complete post-terminal closure.
RQ5	Supported within the evaluated seams	Speech ingress, planner admission, deterministic dispatch, typed feed-back, and planner-dialogue relay remained inspectable. The F13 semantic counterexamples show that ownership separation must be combined with exact target, postcondition, and closure validation.

Surface fluency was not a reliable proxy for planner correctness. The alternate-model run passed the main questionnaire while producing substantially more invalid planner output and recovery failures. Likewise, Contract 6 success and a terminal speech event could coexist with stale KB relations or incomplete reporting. The evaluation therefore treats trajectory completeness, semantic target preservation, state correctness, and exactly-once speech as independent dimensions.

Chapter 8

Discussion

8.1 Architectural Findings

The implementation and runtime traces show that modular planning can be inspected at boundaries that are absent from a prompt-to-action design. The planner does not own robot execution, speech output, or raw perception. It receives structured requests and returns structured plans or dialogue acts. Direct atomic execution was retained as an orchestrator control path, but it was not treated as a matched experimental baseline because its capability scope differs from multi-step planner mediation.

The boundary between `planner_llm` and `nao_orchestrator` is especially important. The planner proposes work over the reviewed skill registry, while the orchestrator independently validates and dispatches it. This permits the same planning path to be exercised with real adapters, deterministic fake skills, or unavailable skills without changing planner ownership. Execution feedback can then return as a typed planning input rather than an unstructured backend error. The architecture therefore constrains where model output may influence the robot, although the semantic audit shows that separation alone does not guarantee correct target selection or post-action state.

This result is consistent with research that questions the reliability of unconstrained language-model planning and instead grounds model proposals in available actions or callable APIs (Huang et al. 2022b; Ahn et al. 2022; Liang et al. 2022; Valmeekam et al. 2023). The implemented contribution is a modular NAO integration in which request admission, skill availability, execution, feedback, and user-facing closure remain attributable to named owners and interfaces.

8.2 Harness Engineering and Failure Attribution

The development and validation evidence shows that harness engineering is necessary when evaluating a language model connected to a live robot environment. Several failures initially appeared to be dialogue or planning problems but originated elsewhere. A complete KB

fixture could be projected without producing an authoritative target selection; a successful action result could leave mutually inconsistent location facts; and a complete ROS trajectory could preserve the wrong object or person roles. Prompt inspection alone cannot distinguish these cases.

The runtime harness establishes the evidence chain needed for that distinction. It verifies source and image identity, lifecycle and QoS readiness, fixture insertion, KB state, dialogue isolation, and action-server availability before injection. During a case, it relates the emitted utterance to route selection, admitted planner context, selected entities, the candidate plan, dispatched skills, execution feedback, KB postconditions, structured reporting, and speech. It also preserves failures and isolated reruns rather than replacing them with a later successful probe. Together, these checks determine which claims a run can support.

Table 8.1 pairs representative runtime symptoms with the boundaries that must be checked to identify their cause.

Table 8.1: Representative failure shapes and the architectural boundaries needed to diagnose them.

Observed symptom	Relevant boundary	Interpretation
Complete fixture followed by unnecessary clarification	Grounded context, authoritative selection, and planner admission	The presence of an entity in context does not prove that the executable target set was preserved.
Person, support, or room used as a movable-object target	KB class, interaction role, registry argument, and semantic oracle	Entity type and task role must remain distinct through selection, planning, execution, and reporting.
Successful delivery followed by stale source-location facts	Skill result, declared KB effects, and post-query verification	Action completion and world-state correctness are separate validation claims.
Multi-object execution reported as one completed object	Execution-history aggregation and <code>report_result</code>	User-facing closure must cover all selected members for which execution evidence exists.
Complete ROS trajectory for the wrong selected set	Case oracle and target-set comparison	Interface traversal establishes observability, not semantic task correctness.

The implemented harness remains tailored to this stack. A reusable environment-adaptive harness would require explicit adapters for readiness, fixture and state control, interaction ingress, semantic oracles, failure injection, and evidence export. The development history suggests that such a harness would reduce diagnostic ambiguity and limit the tendency to treat integration faults as prompt faults. The thesis does not quantify the engineering effort that would have been avoided, so this remains a design implication rather than a measured productivity claim.

8.3 Prompt Packs and Skill Contracts as Runtime Policy

The chatbot and planner prompt packs behaved as versioned runtime policy rather than informal prose. They defined the model’s role, route labels, planning constraints, and output expectations, while the Python builders supplied stage-specific structure and the current request context. Treating these packs as configuration made their revision, hashing, and bounded comparison part of the run tuple. It also prevented a local wording workaround from becoming an unrecorded change to dialogue, grounding, planning, or reporting behaviour.

The evidence further shows why prompt changes should follow failure attribution. A prompt mutation cannot restore a fact that never entered the KB, repair an expired fixture, provide a missing canonical identifier, or remove a stale postcondition. Conversely, a complete grounded context does not ensure that the model will use the correct entities or return valid planner JSON. The bounded prompt-revision procedure therefore evaluates route safety, KB use, execution admission, reporting, and duplicate-speech holdouts before accepting a policy change. This is a narrower claim than general prompt optimisation: it is regression control for two runtime-critical prompt packs.

The reviewed skill registry provides the complementary capability policy. ROS action and service definitions specify transport-level fields, but they do not by themselves explain to a language model when a skill is applicable, what evidence constitutes success, which effects are expected, or which failures may require clarification or replanning. Contract 4 adds planner-facing parameters, grounding requirements, declared preconditions, expected effects, observable success evidence, known failures, recovery guidance, availability, and executor mapping. The same entry informs prompt construction, deterministic admission, dispatch, and result interpretation.

This design resembles the broader principle behind affordance-constrained planning and API-grounded policy generation (Ahn et al. 2022; Liang et al. 2022). In this stack, however, the registry remains a reviewed declaration rather than a learned affordance model or a formal proof system. It narrows the proposal surface and makes unsupported work rejectable, but it cannot guarantee that a semantically valid combination of registered skills has been selected. The F11 invalid-output pressure and the F13 target-role failures demonstrate these distinct limits.

8.4 Planner Behaviour and Feedback

The planner design is strongest when the task can be represented as a sequence of available skill contracts. Atomic commands, object search, navigation, and report-generation tasks map naturally to the registry. Composite instructions are harder because they require the planner to preserve partial order, carry evidence between steps, and decide whether later work remains valid after an earlier failure. These findings support RQ1 for the frozen task families rather than for unrestricted natural-language planning.

The deterministic failure profiles showed that recovery quality depends on both feedback lineage and the availability of a semantically different valid plan. Navigation fail-once cases produced newer plan versions and truthful closure in the strongest historical sweep. Pick and blocked-delivery cases were less stable, and some traces reached a safe stop without complete post-terminal closure. Feedback therefore enabled bounded replanning and clarification, as proposed by interaction-oriented planning approaches (Huang et al. 2022c; Yao et al. 2022), but the evidence does not support a general recovery guarantee for RQ4.

The deterministic guards contain malformed model output and do not form an alternative planner. Route checks, schema validation, registry lookup, argument checks, lineage comparison, and deduplication reject or normalise a bounded set of invalid outputs. Planner compatibility repairs run only after the model path and validation retry, remain visible in the trace, and return typed failure when their conditions do not hold. This order preserves the LLM as the source of planning decisions while preventing malformed output from becoming direct robot work.

8.5 Grounding, Identity, and World State

Grounding remains a central limitation. The architecture distinguishes `grounded_context`, the detector-derived scene summary, and raw KB evidence because each has a different role. The chatbot needs a bounded symbolic projection, the planner needs actionable identifiers and roles, and operators need source evidence with freshness and provenance. The F13 semantic audit showed that presence in the projection is insufficient when target roles are not preserved through selection, planning, dispatch, and reporting.

The gold-apple handoff provides a narrower example. The fixture exposed the apple and named recipient, but the authoritative selected set was empty and admission ended in clarification. This was a truthful stop because no unsupported placement was executed, yet it was not a successful use of complete context. The case separates context availability from target resolution and shows why RQ2 must be evaluated across the whole handoff rather than by searching a prompt trace for an entity name.

Person handling exposes a related semantic requirement. Tracked-person state and KnowledgeCore facts may identify a named individual, but downstream projections must retain whether that entity is a recipient, a gaze target, or another human participant. A person must not be promoted to a movable object or location because a malformed relation places it near an object. The same requirement applies to tables, rooms, and other supports that can anchor an instruction without becoming members of the requested object set. Typed KB classes, explicit task roles, and case-level forbidden-target checks therefore complement the compact projection.

Stable KB insertion probes separated fact availability from model use. Name, colour, support, and count queries passed in broad runs, while grouped post-effect queries exposed stale location relations after apparent delivery success. This confirms the value of symbolic state for bounded grounding, as also motivated by robot knowledge frameworks (Tenorth et al. 2013;

Tenorth et al. 2017; Saxena et al. 2014). It also shows that planning-time state and verified execution-time state must remain separate. A fact used to choose a plan cannot establish that the plan later changed the world as intended.

8.6 Execution Evidence and Truthful Closure

Contract 6 makes the skill result the owner of fresh execution evidence. For state-changing skills, the result may also declare KB effects that the orchestrator applies through `kb_skills` and verifies through a later query. This creates two related but distinct checks: whether the action server returned a typed success and whether the expected symbolic postcondition became observable. The stale-location findings show why both are required.

Reporting adds a further evidence transformation. The planner may request `report_result`, but it cannot predeclare the result text of a sensing, navigation, motion, or manipulation step. The orchestrator accumulates typed results, and the reporting path derives its structured outcome from successful and failed execution evidence. A valid terminal report must cover the selected members, preserve recipients and anchors as roles rather than completed objects, and avoid claiming effects whose postconditions were not verified. Exactly-once speech is necessary at this boundary, but a single utterance can still be semantically incomplete.

Deterministic fake skills were valuable because they exposed success, failure, replanning, KB effects, and reporting under controlled policies. Their results support software-level claims about admission, lineage, dispatch, recovery, and symbolic state. They do not establish physical navigation accuracy, grasp reliability, or human-robot safety. This distinction keeps the fake-skill evidence useful without treating simulation semantics as embodied performance.

8.7 ROS 2 and HRI Modularity

The stack is partly robot-agnostic and partly NAO-specific. The planner request, planner output, execution feedback, dialogue act, and compact grounding contracts are not inherently tied to NAO. Reuse on another ROS-compatible platform would require an equivalent reviewed skill registry and adapters that produce the same typed evidence. Speech execution, replay motion, posture control, head motion, and look-at behaviour remain dependent on NAO-specific packages or adapters, so cross-platform portability was not demonstrated.

ROS 2 distribution and reusable HRI interfaces introduce integration costs as well as useful boundaries (Quigley et al. 2009; Macenski et al. 2022; Mohamed et al. 2020). Lifecycle state, QoS, topic remapping, per-person dialogue state, action availability, and transform ownership can each invalidate a run without appearing as a planner exception. The preflight and evidence-bundle requirements in Chapter 6 are therefore part of preserving the public interaction contract rather than incidental deployment checks.

The ownership split also prevents convenient shortcuts from becoming architectural drift. `dialogue_manager` retains dialogue lifecycle and final speech realization; `chatbot_llm` owns

user-facing language, route selection, and candidate planner ingress; `planner_llm` owns plan generation and feedback-driven planning responses; `nao_orchestrator` owns admission, deterministic dispatch, feedback, and planner-dialogue relay; `kb_skills` owns KnowledgeCore transport; and runtime skills own fresh evidence for their effects. This separation makes failures more numerous as observable categories, but fewer of them can cross silently from model output to robot action.

8.8 Validation Implications

The difference between the F13 trajectory result and semantic audit is the principal methodological finding. A case can contain speech ingress, planner admission, execution feedback, a terminal marker, and final speech while still acting on the wrong targets. Full user-turn injection remains necessary for route correctness, dialogue ownership, and exactly-once speech. Planner and action-level probes remain useful for isolating plan admission and skill guards. Neither is sufficient without an oracle that compares requested, selected, planned, executed, and reported members and then verifies the resulting KB state.

The runtime-review index is useful for operational triage, but it remains secondary to explicit denominators and acceptance rules. The earlier trajectory assessment and later semantic cap describe different evidence depths rather than a uniform performance decline. Reporting both exposes the stronger acceptance criterion instead of suppressing a safety-relevant finding to preserve a headline score. The numerical rates determine how often the declared contracts held in the frozen cases; they do not estimate unrestricted task success or physical robot performance.

The research questions therefore receive different kinds of support. RQ1 concerns plan coherence over the frozen skill families. RQ2 requires entity and role consistency from source evidence through post-state verification. RQ3 concerns rejection of schema-invalid, registry-invalid, argument-invalid, and lineage-invalid work, not the elimination of all model errors. RQ4 depends on injected failures and traceable plan-version transitions. RQ5 is supported most directly by the named ownership boundaries and the harness evidence needed to inspect them. This division keeps the interpretation aligned with scenario-based validation rather than a broader claim of general robotic autonomy.

Chapter 9

Limitations and Future Work

9.1 Current Limitations

The current system has several limitations. The physical implementation is centred on NAO, so cross-robot portability is architectural rather than fully proven. LLM planning remains sensitive to prompt design, model availability, token limits, and JSON schema adherence. The validation strategy is scenario-based and cannot prove exhaustive safety. Perception and grounding quality also constrain planner performance because a correct planner cannot reliably act on objects that are missing, stale, or ambiguously represented in the world state.

The recovered evidence is qualified by mutable or multi-image runtime provenance, so it does not yet define one final frozen aggregate. The semantic audit also found incomplete target selection for the gold-apple handoff, stale KB postconditions after some grouped deliveries, and incomplete closure in selected failure profiles. Deterministic fake skills validate supervision and symbolic effects but do not establish physical navigation or manipulation performance. Detector-enabled grounding remains separate because the principal semantic profile intentionally disables detector scoring.

9.2 Future Work

Future validation should freeze one clean source revision, nested revision, container digest, model configuration, prompt and registry hashes, fixtures, and questionnaires before running the complete profile matrix. The deterministic policy ladder should be followed by repeated compact robustness cases and a separately scored detector profile. Cross-run comparison should use the hash-addressed bundles and semantic metrics defined in Chapter 6.

Architectural extensions include richer grounding through a persistent world model, broader skill-registry generation from reviewed contracts, safer composite execution, and cross-platform validation on another ROS-compatible robot or simulator. A longer-term extension would track uncertainty, object identity, and temporal freshness explicitly while preserving the compact planner-facing projection.

Chapter 10

Conclusion

This thesis has presented the design, integration, and validation of a modular language-model-enabled interaction and execution stack for the NAO robot. The contribution extends beyond a planner node: it combines dialogue lifecycle management, semantic routing, compact symbolic grounding, structured task planning, deterministic orchestration, real and simulated skills, execution feedback, and evidence-based user communication. Explicit runtime contracts keep those responsibilities attributable to named ROS 2 interfaces.

The implementation demonstrates how a chatbot can route execution-looking turns to a planner, how the planner can produce structured plans over a skill registry, and how an orchestrator can act as the execution boundary between generated plans and robot or fake-skill adapters. The runtime contracts make the system more auditable than a direct prompt-to-action design because planner requests, grounded context, generated plans, execution feedback, and dialogue acts can be captured and checked independently.

The runtime evidence supports the usefulness of deterministic fake skills for controlled plan, failure, and symbolic-effect validation. The planner produced coherent multi-step plans for the frozen motion, navigation, reporting, and grouped-delivery cases, while registry and orchestrator validation rejected malformed or semantically invalid work before dispatch. Feedback supported navigation replanning and truthful failure handling in bounded cases, but manipulation recovery and terminal closure were less consistent.

Compact grounding supported stable next-turn reference to injected objects and relations. It did not guarantee that entity roles, selected members, or post-action state remained correct throughout execution. The difference between trajectory completion and semantic correctness was therefore material: one broad questionnaire passed its ROS trajectory checks while a stronger audit found wrong target roles, incomplete reports, stale KB effects, and dialogue-to-execution leakage. Exact target-set, postcondition, and closure validation are required in addition to interface-level observability.

Overall, the work supports a measured conclusion: language-model-based dialogue and planning can participate in a robot stack when symbolic context, reviewed skills, deterministic orchestration, and typed feedback constrain their influence on execution. The approach does not remove the need for runtime validation, but it provides explicit boundaries and

reproducible evidence with which that validation can be performed.

Appendix A

Full Runtime Contracts

The full runtime contracts are described in Chapter 4. This appendix is reserved for additional machine-readable schemas when the final validation artefacts are frozen.

A.1 Runtime Skill Inventory

Table A.1 gives the compressed runtime projection used by the current planner and orchestrator profiles. The values are taken from the shared skill registry and are shortened only to fit the appendix. The machine-readable registry and the generated runtime projections are frozen with the source revision of each validation bundle. The inventory includes all sixteen planner-visible runtime skills, including the non-physical communication skill **ask_user**. Its inclusion does not move final wording or speech ownership out of the dialogue layer described by Contract 8.

Table A.1: Compressed runtime skill inventory and registry values.

Skill and category	Required parameters	Expected effect or evidence	Known failure modes	Executor and status
scan perception	none	refreshed scene evidence and typed scan payload	no_person, objects_only, backend_unavailable, unexpected_payload	orchestrator.scan; first-party
find_object perception	target	object or person evidence is returned	not_found, ambiguous, backend_unavailable	fake.find_object; fake
navigate_to navigation	target	robot arrives at the target location	path_blocked, unknown_location, safety_disabled	fake.navigate_to; fake
walk_to navigation	none	short local walking segment is executed or dry-run	invalid_local_goal, safety_disabled, path_blocked, naoqi_move_failed	orchestrator.walk_to; hybrid
bring_object manipulation	target, recipient	object is delivered to recipient or destination	acquisition_failure, navigation_failure, delivery_blocked, recipient_unavailable	fake.bring_object; fake
place_object manipulation	target, destination	object is released and a support relation is added	destination_unavailable, invalid_support, no_held_object, placement_failed	fake.place_object; fake
pick_object manipulation	target	robot holds the object and updates its support relation	object_unavailable, unreachable, already_held, grasp_failed	fake.pick_object; fake
perform_motion embodiment	object	robot posture or head orientation changes	unsupported_motion, dispatch_failed	orchestrator.perform_motion; first-party
look_at attention	none	robot gaze is redirected or reset	missing_target, dispatch_failed	orchestrator.look_at; first-party
kb_add knowledge	statements	requested predicates are added through KnowledgeCore	missing_statements, kb_service_unavailable, kb_mutation_rejected	orchestrator.kb_add; first-party
kb_revise knowledge	statements	requested predicates are revised through KnowledgeCore	missing_statements, kb_service_unavailable, kb_mutation_rejected	orchestrator.kb_revise; first-party
kb_remove knowledge	statements	requested predicates are removed through KnowledgeCore	missing_statements, kb_service_unavailable, kb_mutation_rejected	orchestrator.kb_remove; first-party
report_result communication	none	user receives a result summary through robot speech	missing_summary, say_dispatch_failed	orchestrator.report_result; first-party
ask_user communication	none	structured clarification request obtains missing user information	missing_prompt, dialogue_relay_unavailable	orchestrator.ask_user; first-party
wave_greet social motion	none	robot performs a greeting wave gesture	motion_unavailable, safety_disabled, unexpected_payload	orchestrator.wave_greet; hybrid
inspect_area perception	target	scene evidence for the target area is returned	ambiguous, area_unknown, backend_unavailable	fake.inspect_area; fake

The inventory separates the planner-facing name from the executor mapping. In the display column, `orchestrator.*` abbreviates `nao_orchestrator.*`, and `fake.*` abbreviates `fake_skills.*`. The table also records whether failure injection is supported in the current fake profile, which is used by the multi-step deterministic fake-skill cases in Chapter 6. The table does not imply that every hybrid or fake entry is enabled in the primary scored launch profile.

Appendix B

Launch Commands

The command below starts the compact planner-enabled simulator profile. Scored validation bundles record the complete launch argument tuple described in Chapter 6.

```
1 ros2 launch nao_chatbot nao_chatbot_sim_demo.launch.py \  
2   start_planner_llm:=true \  
3   chatbot_planner_mode_enabled:=true \  
4   planner_llm_think:=false
```

Appendix C

Validation Log Template

Table C.1 shows the fields used to index one validation case. Machine-readable traces and run manifests remain the authoritative evidence for aggregate results.

Field	Value
Scenario ID	<i>S1–S8</i>
User command	<i>Paste command</i>
Model	<i>Model name</i>
Planner request	<i>Path or pasted payload</i>
Planner output	<i>Path or pasted payload</i>
Execution feed-back	<i>Path or pasted payload</i>
Dialogue act	<i>Path or pasted payload</i>
Outcome	<i>success / failure / recovered / clarification</i>
Notes	<i>Observations</i>

Table C.1: Validation log template.

Bibliography

- Ahn, Michael, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Chuyuan Fu, Keerthana Gopalakrishnan, Karol Hausman, et al. (2022). “Do As I Can, Not As I Say: Grounding Language in Robotic Affordances”. In: *arXiv preprint arXiv:2204.01691*.
- Bastianelli, Emanuele, Domenico D. Bloisi, Roberto Capobianco, Guglielmo Gemignani, Luca Iocchi, and Daniele Nardi (2013). “Knowledge representation for robots through human-robot interaction”. In: *arXiv preprint arXiv:1307.7351*.
- Brohan, Anthony, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, et al. (2023). “RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control”. In: *arXiv preprint arXiv:2307.15818*.
- Driess, Danny, Fei Xia, Mehdi S. M. Sajjadi, Corey Lynch, Aakanksha Chowdhery, Brian Ichter, Ayzaan Wahid, Jonathan Tompson, Quan Vuong, Tianhe Yu, et al. (2023). “PaLM-E: An Embodied Multimodal Language Model”. In: *arXiv preprint arXiv:2303.03378*.
- Erol, Kutluhan, James Hendler, and Dana S. Nau (1994). “HTN Planning: Complexity and Expressivity”. In: *Proceedings of the Twelfth National Conference on Artificial Intelligence*. AAAI Press, pp. 1123–1128.
- Fong, Terrence, Illah Nourbakhsh, and Kerstin Dautenhahn (2003). “A survey of socially interactive robots”. In: *Robotics and Autonomous Systems* 42:3–4, pp. 143–166.
- Georgievski, Ilche and Marco Aiello (2014). “An Overview of Hierarchical Task Network Planning”. In: *arXiv preprint arXiv:1403.7426*.
- Goodrich, Michael A. and Alan C. Schultz (2007). “Human-robot interaction: a survey”. In: *Foundations and Trends in Human-Computer Interaction* 1.3, pp. 203–275.
- Gouaillier, David, Vincent Hugel, Pierre Blazevic, Chris Kilner, Jérôme Monceaux, Pascal Lafourcade, Brice Marnier, Julien Serre, and Bruno Maisonnier (2008). “The NAO humanoid: a combination of performance and affordability”. In: *arXiv preprint arXiv:0807.3223*.
- Grassi, Lucrezia, Carmine Tommaso Recchiuto, and Antonio Sgorbissa (2021). “Knowledge triggering, extraction and storage via human-robot verbal interaction”. In: *arXiv preprint arXiv:2104.11170*.
- Huang, Wenlong, Pieter Abbeel, Deepak Pathak, and Igor Mordatch (2022a). “Language Models as Zero-Shot Planners: Extracting Actionable Knowledge for Embodied Agents”. In: *International Conference on Machine Learning*.

- Huang, Wenlong, Pieter Abbeel, Deepak Pathak, and Igor Mordatch (2022b). “Language Models as Zero-Shot Planners: Extracting Actionable Knowledge for Embodied Agents”. In: *arXiv preprint arXiv:2201.07207*.
- Huang, Wenlong, Fei Xia, Ted Xiao, Harris Chan, Jacky Liang, Pete Florence, Andy Zeng, et al. (2022c). “Inner Monologue: Embodied Reasoning through Planning with Language Models”. In: *arXiv preprint arXiv:2207.05608*.
- Kim, Moo Jin, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, et al. (2024). “OpenVLA: An Open-Source Vision-Language-Action Model”. In: *arXiv preprint arXiv:2406.09246*.
- Li, Junnan, Dongxu Li, Silvio Savarese, and Steven Hoi (2023). “BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models”. In: *arXiv preprint arXiv:2301.12597*.
- Liang, Jacky, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng (2022). “Code as Policies: Language Model Programs for Embodied Control”. In: *arXiv preprint arXiv:2209.07753*.
- Liu, Bo, Yuqian Jiang, Xiaohan Zhang, Qiang Liu, Shiqi Zhang, Joydeep Biswas, and Peter Stone (2023a). “LLM+P: Empowering Large Language Models with Optimal Planning Proficiency”. In: *arXiv preprint arXiv:2304.11477*.
- Liu, Haotian, Chunyuan Li, Qingyang Wu, and Yong Jae Lee (2023b). “Visual Instruction Tuning”. In: *arXiv preprint arXiv:2304.08485*.
- Macenski, Steve, Tully Foote, Brian Gerkey, Chris Lalancette, and William Woodall (2022). “Robot Operating System 2: Design, architecture, and uses in the wild”. In: *Science Robotics* 7.66, eabm6074.
- McDermott, Drew, Malik Ghallab, Adele Howe, Craig Knoblock, Ashwin Ram, Manuela Veloso, Daniel Weld, and David Wilkins (1998). *PDDL: The Planning Domain Definition Language*. Tech. rep. CVC TR-98-003/DCS TR-1165. Yale Center for Computational Vision and Control.
- Mohamed, Youssef and Séverin Lemaignan (2020). “ROS for Human-Robot Interaction”. In: *arXiv preprint arXiv:2012.13944*.
- Nau, Dana S., Tsz-Chiu Au, Okhtay Ilghami, Ugur Kuter, J. William Murdock, Dan Wu, and Fusun Yaman (2003). “SHOP2: An HTN Planning System”. In: *Journal of Artificial Intelligence Research* 20, pp. 379–404.
- Open Robotics (2025). *Robot Operating System 2 Documentation*. Accessed as project background documentation.
- Quigley, Morgan, Ken Conley, Brian Gerkey, Josh Faust, Tully Foote, Jeremy Leibs, Rob Wheeler, and Andrew Y. Ng (2009). “ROS: an open-source Robot Operating System”. In: *ICRA Workshop on Open Source Software*.
- ROS4HRI Community (2025). *ROS4HRI: ROS for Human-Robot Interaction*. Accessed as project background documentation.
- Saxena, Ashutosh, Ashesh Jain, Ozan Sener, Aditya Jami, Dipendra K. Misra, and Hema S. Koppula (2014). “RoboBrain: Large-scale knowledge engine for robots”. In: *arXiv preprint arXiv:1412.0691*.

- Shinn, Noah, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao (2023). “Reflexion: Language Agents with Verbal Reinforcement Learning”. In: *arXiv preprint arXiv:2303.11366*.
- Tenorth, Moritz and Michael Beetz (2013). “KnowRob: A knowledge processing infrastructure for cognition-enabled robots”. In: *The International Journal of Robotics Research* 32.5, pp. 566–590.
- (2017). “Representations for robot knowledge in the KnowRob framework”. In: *Artificial Intelligence* 247, pp. 151–169.
- Valmeekam, Karthik, Matthew Marquez, Sarath Sreedharan, and Subbarao Kambhampati (2023). “On the Planning Abilities of Large Language Models: A Critical Investigation”. In: *arXiv preprint arXiv:2305.15771*.
- Yao, Shunyu, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao (2022). “ReAct: Synergizing Reasoning and Acting in Language Models”. In: *arXiv preprint arXiv:2210.03629*.